

PZ-SIM800C GSM-GPRS 模块开发手册

本手册我们将向大家介绍 PZ-SIM800C GSM-GPRS 模块及其在普中 F407 最小系统&天马&麒麟开发板上的使用。本章分为如下几部分内容：

- 1 PZ-SIM800C GSM-GPRS 模块介绍
- 2 硬件设计
- 3 软件设计
- 4 实验现象

1 PZ-SIM800C GSM-GPRS 模块介绍

1.1 特性参数

PZ-SIM800C GSM-GPRS 模块是深圳普中科技推出的一款高性能工业级 GSM/GPRS 模块（开发板）。PZ-SIM800C GSM-GPRS 模块板载 SIMCOM 公司的工业级四频 GSM/GPRS 模块：SIM800C，工作频段四频：850/900/1800/1900MHz，可以低功耗实现语音、SMS(短信)、MMS(彩信)、蓝牙数据信息的传输。

PZ-SIM800C 模块支持 RS232 串口和 LVTTL 串口(即支持 3.3V/5V 系统)，并带硬件流控制，支持 5V~24V 的超宽工作范围，使得本模块可以非常方便的与您的产品进行连接，从而给您的产品提供包括语音、短信、彩信、蓝牙和 GPRS 数据传输等功能。

PZ-SIM800C GSM-GPRS 模块的特性如图所示：

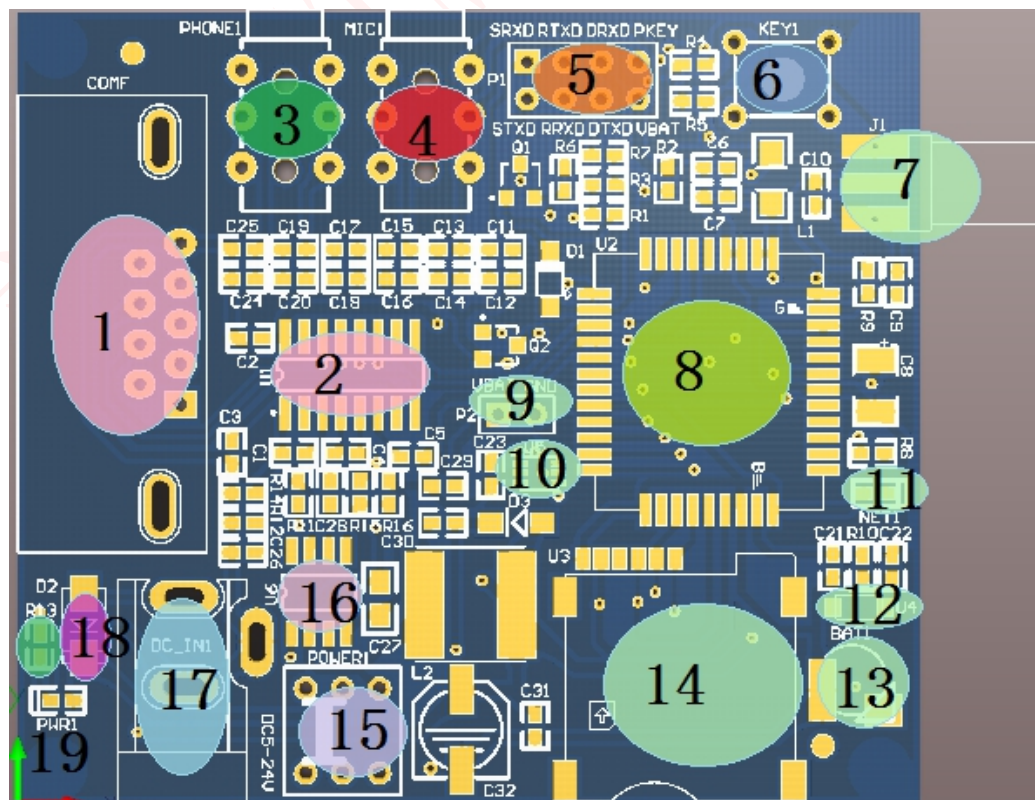
项目	说明
通信接口	RS232 串口/LVTTL 串口、 支持 AT 命令控制（3GPP TS 27.007，27.005 和 SIMCOM 增强 AT 命令集）、 支持 RTS/CTS 硬件流控制、 支持符合 GSM 07.10 协议的串口复用功能 支持从 1200bps 到 115200bps 的自动波特率检测功能、 支持软件升级
语音接口	3.5mm 耳机+麦克风座
天线接口	SMA 接口，自带 GSM（850M/900M/1800M/1900M）专用小辣椒天线，2.4G 蓝牙陶瓷天线
电源接口	DC005-2.1mm 直流电源座
SIM 卡接口	支持 1.8/3V SIM 卡
工作频段	四频：GSM850M、EGSM900M、DCS1800M、PCS1900M 自动搜索 4 个频段
发射功率	Class4(2W)：GSM850M、EGSM 900M Class1(1W)：DCS1800M、PCS1900M
GPRS 连接特性	GPRS 时隙缺省为等级 12 GPRS 时隙 class8, 10, 12 可选 GPRS 移动台等级 B
工作温度	-40℃~+85℃
模块尺寸	62mm*54.38mm (仅PCB部分)
电源供电	DC5~24V

I/O 电平	Voh(max) 2.8V、Vol(min) 0V（对于通信接口（即：STXD/SRXD/PKEY 等接口），可以兼容 3.3V/5V 单片机系统）
功耗	12~90mA@1.2V
GPRS 数据特性	下行传输速率：最大 85.6kbps 上行传输速率：最大 85.6kbps 编码格式：CS-1、CS-2、CS-3 和 CS-4 支持通常用于 PPP 连接的 PAP（密码验证协议）协议 内嵌 TCP/IP 协议 支持分组广播控制信道（PBCCH）
音频特性	支持半速率、全速率、增强型全速率、自适应多速率等编码模式 支持回声消除功能 支持噪声抑制功能
短信（SMS）	支持 MT/MO/CB/TEXT 和 PDU 模式 短信存储设备：SIM 卡
其他功能	彩信、DTMF、TTS、录音、蓝牙串口
通信录管理	支持类型：SM/FD/LD/RC/ON/MC
SIM 应用工具包	支持 SAT class3、GSM 11.14 Release 99
实时时钟（RTC）	支持，并带后备电池（XH414H-IV01E）供电
软件升级	通过全功能串口升级软件

1.2 模块说明

1.2.1 模块简介

PZ-SIM800C GSM-GPRS 模块是深圳普中科技开发的一款高性能工业级 GSM/GPRS 模块（开发板），功能完善，尤其适用于需要语音/短信/GPRS 数据/蓝牙通信服务的各种领域，其资源图如图所示：



从图中可以看出，PZ-SIM800C 模块不但外观漂亮，而且功能齐全，模块尺寸（不算天线部分）为 62mm*54.38mm，并带有安装孔位，非常小巧，并且利于安装，可方便应用于各种产品设计。

PZ-SIM800C GSM-GPRS 模块（开发板）板载资源如下：

标号	名称
1	RS232串口
2	SP3232芯片
3	耳机接口
4	麦克风（MIC）接口
5	RS232 选择和PKEY 引出接口
6	开机/关机按键
7	SMA天线接口
8	SIM800C模块
9	锂电池接口
10	SMF05C ESD保护
11	网络状态指示灯
12	2.4G陶瓷蓝牙天线
13	RTC后备电池
14	自弹式Micro SIM 卡座
15	电源开关
16	MP2303芯片
17	电源输入接口
18	电源防反接保护二极管
19	电源指示灯

PZ-SIM800C GSM-GPRS 模块（开发板）采用工业级标准设计，特点包括：

- 1) 板载 RS232 串口（支持硬件流控制），方便与 PC/工控机等设备连接；
- 2) 板载 3.5mm 耳机和麦克风座，方便进行语音通信开发；
- 3) 板载高效同步降压电路，转换效率高达 90%，支持超宽电压工作范围（5~24V），非常适合工业应用；
- 4) 板载电源防反接保护，SIM 卡 ESD 保护，保护功能完善；
- 5) 板载 RTC 后备电池（XH414H-IV01E），无需担心掉电问题；
- 6) 板载小辣椒天线和陶瓷天线，能有效提高信号接收能力；
- 7) 采用国际 A 级 PCB 料，沉金工艺加工，稳定可靠；
- 8) 采用全新元器件加工，纯铜镀金排针，坚固耐用；
- 9) 人性化设计，各个接口都有丝印标注，使用起来一目了然；接口位置设计安排合理，方便顺手。
- 10) PCB 尺寸为 62mm*52.5mm，并带有安装孔位，小巧精致；

1.2.2 模块硬件资源详解

(1) GSM 模块(U2)

PZ-SIM800C 所选择的 GSM 模块为 SIMCOM(希姆通)公司的 SIM800C 模块,该模块为 SIMCOM 公司推出的一款紧凑型产品,完全采用 SMT 封装形式,其性能稳定,外观精巧,性价比高。SIM800C 采用工业标准接口,工作频率为 850/900/1800/1900Mhz,内嵌 TCP/IP 协议,可以低功耗实现语音, SMS(短信)、MMS(彩信)、蓝牙数据信息的传输。

(2) RTC 后备电池(BAT1)

PZ-SIM800C 板载了 RTC 后备电池,采用 SIMCOM 公司推荐的 XH414H-IV01E 作为 SIM800C 模块的 RTC 后备电池, XH414H 具有尺寸小,容量大,可反复充放电的特点,能维持 RTC 的长时间掉电运行。

(3) 麦克风(MIC1)/耳机接口(PHONE1)

PZ-SIM800C 板载一个 3.5mm 麦克风接口 (MIC1) 和一个 3.5mm 耳机接口 (PHONE1), 用于实现语音通话功能。

(4) 功能选择接口(P1)

该接口 (P1, 即 RS232 选择和 PKEY 引出接口) 用于选择 RS232 串口连接到 SIM800C 的通信端口, 或者调试 (Debug) 端口, 以及设置 SIM800C 上电自动开机。

其中 STXD 和 SRXD, 是 SIM800C 的数据通信串口, 我们默认发送的 AT 指令以及数据等, 都是通过这两个端口。而 DTXD 和 DRXD, 则是调试串口, 主要是软件升级时使用, 一般用不到, 不过我们也留出了, 方便大家后续升级使用。RTXD 和 RRXD 则是 RS232 串口经过 SP3232 芯片转换后的串口端口。VBAT 是供电引脚, 当与 PKEY 连接时, 模块上电自动开机。

另外, STXD 和 SRXD 做了兼容性处理, 支持 LVTTTL 电平 (即 3.3V/5V) 的单片机系统, 可以直接将 STXD 和 SRXD 与单片机系统的 RXD 和 TXD 连接, 实现与 SIM800C 的通信。

模块默认是将 RS232 串口连接在 SIM800C 的通信端口 (即 STXD 与 RRXD 连接, SRXD 与 RTXD 连接)。

(5) RS232 串口(COMF)

该接口（COMF）为 RS232 串口，用于连接 PC 或工控机等设备的串口，实现对 SIM800C 的控制，PZ-SIM800C 模块选择 SP3232 作为电平转换芯片，实现 SIM800C 的 RS232 串口。

RS232 串口通过 P1 端口，选择连接到 SIM800C 的通信串口，还是调试串口，默认连接的是通信串口。

（6）锂电池接口(P2)

该接口（P2）用于连接外部锂电池，当外部电源切断的时候，可以由锂电池给模块供电，而当外部电源接上时，该接口还可以给锂电池充电（设计电压 4.016V）。

在不使用锂电池的时候，该接口也可以用来给外部供电（4V），或者外部给模块供电（范围：3.4V~4.4V）。

（7）电源输入接口(DC_IN)

该接口（DC_IN）采用 DC005-2.1 座作为模块的直流电源输入接口，支持 DC5~24V 的宽电压输入范围，使得 PZ-SIM800C 模块可以非常方便的与您的设备进行连接。

PZ-SIM800C 模块采用的是 MPS 公司的高效同步降压 IC：MP2303，可以提供非常高的电源转换效率，以及宽电压输入范围。并且 PZ-SIM800C 模块采用了电源防反接保护措施，有效提高模块的可靠性。

（8）电源指示灯(PWR1)

该指示灯（PWR1），是一颗 0603 封装的蓝色 LED，用于指示模块的上电状态，当模块通电的时候该灯亮，否则灭。

（9）电源开关(POWER1)

这是 PZ-SIM800C 模块的总开关，实现外部电源供电的时候，对模块的上电和断电控制。不过需要注意的是：通过 P2 端口供电的时候，该开关不起作用！

（10）Micro SIM 卡座(U3)

该卡座（U3）采用进口高质量自弹式 Micro SIM 卡座，用于安装 Micro SIM 卡。卡座铁壳上面和底板背面标有建议操作图，使用非常简单。

（11）SMA 天线接口(J1)

该接口（J1）采用高质量偏脚 SMA 母座，是 SIM800C 的天线座，用于连

接外部天线。PZ-SIM800C 模块默认都是配送有小辣椒天线，连接该接口，可以有效提高 SIM800C 的信号质量。

(12) 陶瓷天线接口(U4)

该接口（ANT）采用高质量 2.4G 陶瓷天线，用于 SIM800C 的蓝牙天线，空旷地通信有效距离 10~15m 左右。

(13) 开机/关机按键(KEY1)

该按键（KEY1）连接 SIM800C 模块的 PWRKEY 引脚，实现对模块的开关机控制。按下该键 3 秒，然后释放，可以实现开启模块。同样，在模块开启的情况下，按下该键至少 3 秒，即可关闭模块。

PZ-SIM800C 模块上电后，SIM800C 模块默认是关闭的，需要长按（1S 左右）该键，才能开启 SIM800C 模块。

（注意：必须断开 P1 排针 VBAT 与 PKEY 的连接，否则 PWR_KEY 按键无效！！）

(14) 网络状态指示灯(NET1)

该指示灯（NET1）是一颗 0603 封装的红色 LED，用于指示网络状态。其工作状态指示如图所示：

NET 状态	工作状态
熄灭	关机
64ms 亮/800ms 灭	没注册到网络
64ms 亮/3000ms 灭	注册到网络
64ms 亮/300ms 灭	GPRS 通信

通过该指示灯的闪烁情况，我们可以很方便的判断 SIM800C 模块的工作状态。

1.3 模块使用

1.3.1 使用前准备

在使用 PZ-SIM800C 模块之前，我们应先准备一下 4 样东西：

①1 张手机 SIM 卡（未停机，并开通 GPRS 功能（否则不能测试 GPRS 功能））

②1 个外部直流电源(保证能给 SIM800C 提供 2A 电流,推荐使用 12V1A 电源)

③1 根 RS232 串口线 (连接电脑用, 推荐使用 USB 转串口线, 可直接调试模块, 无需外部开发板)

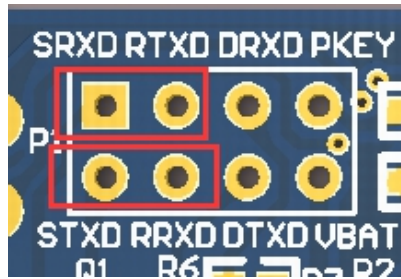
④1 副耳机 (带麦克风功能, 用于测试通话功能)

⑤1 台支持蓝牙的手机设备 (安卓系统)

在备齐上述 4 件套之后, 我们先给模块安装 SIM 卡, 插上耳机和麦克风。这里我们用一根 USB 转串口线来做测试, 所以我们需要先安装 USB 转串口的驱动 (这个要根据你 USB 转串口线的驱动来弄, 一般电脑会自动安装, 如果安装不成功, 请联系您购买这根线的店家), 驱动的安装我们就不多介绍了, 安装驱动后, 我们用 USB 转串口线连接电脑和 PZ-SIM800C 模块, 在设备管理器里面, 我们可以看到被识别出的串口, 我的电脑上被识别为串口 8 (COM8), 如图所示:



不同的电脑, 可能被识别为不同的 COM 号, 使用的时候选择正确即可。之后, 确保 PZ-SIM800C 模块 P1 口的连接方式为: SRXD 连接 RTXD, STXD 连接 RRXD (模块出厂默认已连接), 如图所示:



最后，我们插上电源，给模块供电，并按下电源开关 POWER1（此时电源指示灯 PWR1 会亮起），然后等待 1-3 秒直到模块 NET1 红灯闪烁，实现 SIM800C 模块的自动开机。如果要实现手动开机，请断开 P1 排针 PKEY 与 VBAT 的连接，然后长按 KEY1 按键 1-3 秒直到模块红灯闪烁，这时 SIM800C 就可以实现手动开机。

在模块开机以后，我们可以看到 NET1 指示灯（红色），先是快闪（1 秒/次），表明还没注册到网络，然后在注册到网络后，NET1 慢闪（3 秒 1 次）。这个过程需要数秒~数十秒不等，取决于所在地 GSM 网络的状态。

在看到网络状态指示灯（NET）3 秒 1 闪之后，说明 SIM800C 模块已经连接到网络了，可以开始正常使用了。

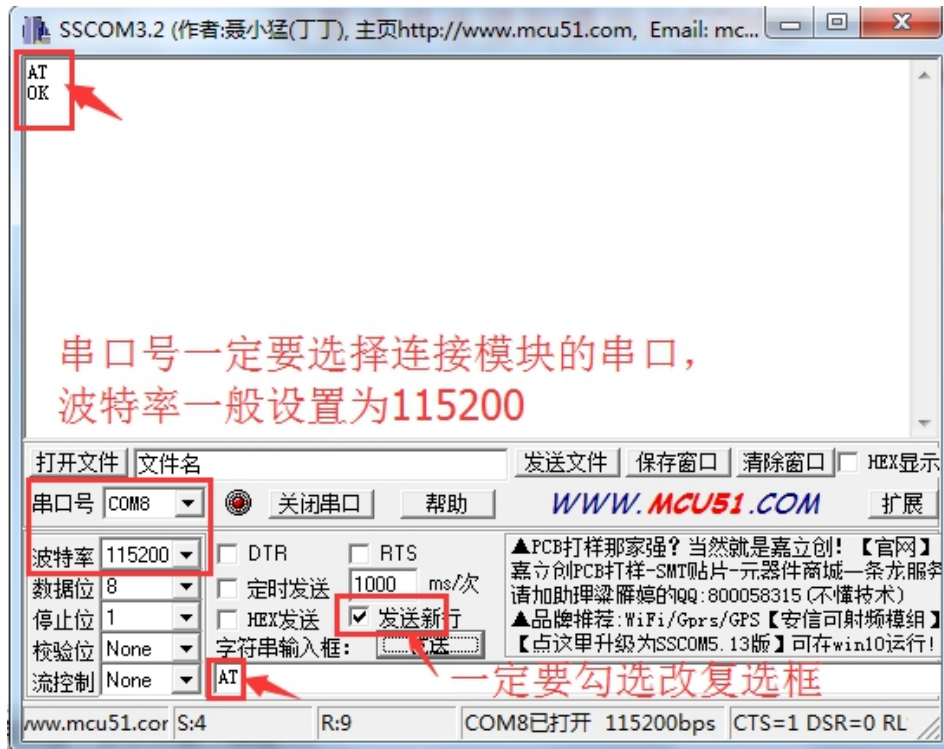
1.3.2 AT 指令简介

AT 即 Attention，AT 指令集是从终端设备(Terminal Equipment, TE)或数据终端设备(DataTerminal Equipment, DTE)向终端适配器(Terminal Adapter, TA)或数据电路终端设备(DataCircuit Terminal Equipment, DCE)发送的。通过 TA, TE 发送 AT 指令来控制移动台(Mobile Station, MS)的功能，与 GSM 网络业务进行交互。用户可以通过 AT 指令进行呼叫、短信、电话本、数据业务、传真等方面的控制。

AT 指令必须以“AT”或“at”开头，以回车（<CR>）结尾。模块的响应通常紧随其后，格式为：<回车><换行><响应内容><回车><换行>。

我们通过串口调试助手来测试一下（可使用一根 USB 转串口线，一端连接电脑一端连接模块 RS232 口），打开：PZ-SIM800C GSM/GPRS 模块配套资料“PZ-SIM800C GSM-GPRS 模块\调试工具\串口调试助手”，选择正确的 COM 号（连接到 PZ-SIM800C 模块的 COM 端口，我电脑是 COM8），然后设置波特率为

115200，勾选发送新行（必选！即该软件自动添加发送回车换行功能），然后发送 AT 到 PZ-SIM800C 模块，如图所示：



上图中我们发送了 1 次 AT 指令，如果第一次看到有乱码，这是因为模块上电后，还没有实现串口同步，在收到第一次数据（不一定要 AT 指令）后，模块会自动实现串口同步（即自动识别出了通信波特率），后续通信就不会出现乱码了。因为 SIM800C 具有自动串口波特率识别功能（识别范围：1200-115200），所以我们的电脑（或设备）可以随便选择一个波特率（不超过识别范围即可），来和模块进行通信，这里我们选择最快的 115200。

从上图中可以看出，我们现在已经和 SIM800C 模块进行通信了，我们通过发送不同的 AT 指令，就可以实现对 SIM800C 的各种控制了。

SIM800C 模块提供的 AT 命令包含符合 3GPP TS 27.005、3GPP TS 27.007 和 ITU-T Recommendation V.25ter 的指令，以及 SIMCOM 自己开发的指令。接下来我们介绍几个常用的 AT 指令：

1, AT+CPIN?

该指令用于查询 SIM 卡的状态，主要是 PIN 码，如果该指令返回：+CPIN:READY, 则表明 SIM 卡状态正常，返回其他值，则有可能是没有 SIM 卡。

在模块出现问题的时候，一定要先发送：AT+CPIN?，查询一下，看看是不

是 SIM 卡和 SIM 卡座没有接触好？如果返回 ERROR，则说明可能是 SIM 卡没接触好，用纱布擦一下 SIM 卡座和 SIM 卡的接触焊盘，然后重装 SIM 卡，重启，一般就可以解决。

2， AT+CSQ

该指令用于查询信号质量，返回 SIM800C 模块的接收信号强度，如返回：
+CSQ: 24,0，表示信号强度是 24（最大的有效值是 31）。如果信号强度过低，则要检查天线是否接好了。

3， AT+COPS?

该指令用于查询当前运营商，该指令只有在连上网络后，才返回运营商，否则返回空，如返回： +COPS:0,0," CHINA MOBILE"，表示当前选择的运营商是中国移动。

4， AT+CGMI

该指令用于查询模块制造商，如返回： SIMCOM_Ltd，说明 SIM800C 模块是 SIMCOM 公司生产的。

5， AT+CGMM

该指令用于查询模块型号，如返回： SIMCOM_SIM800C，说明模块型号是 SIM800C。

6， AT+CGSN

该指令用于查询产品序列号（集 IMEI 号），每个模块的 IMEI 号都是不一样的，具有全球唯一性，如返回： 866104023267696，说明模块的产品序列号是： 866104023267696。

7， AT+CNUM

该指令用于查询本机号码，必须在 SIM 卡在位的时候才可以查询，如返回：
+CNUM:"","13579079501","129",7,4，则表明本机号码为： 13579079501。
另外，不是所有的 SIM 卡都支持这个指令，有个别 SIM 卡无法通过此指令得到其号码。

8， ATE1

该指令用于设置回显模式（默认开启），即模块将收到的 AT 指令完整的返回给发送端，启用该功能，有利于调试模块。如果不需要开启回显模式，则发

送 ATE0 指令即可关闭，这样收到的指令将不再返回给发送端，这样方便程序控制。

9, AT+CGMR

该指令用于查询固件版本序列号，如返回：Revision:1418B05SIM800C24_BT，说明模块的固件版本序列号是 1418B05SIM800C24_BT，flash 大小是 24Mbit、支持蓝牙通信功能。

以上就是我们介绍的几个常用的 AT 指令，当然还有其他一些常用的 AT 指令，比如 ATD/ATA/ATH 等，我们在后面的章节会慢慢介绍。关于 SIM800C 详细的 AT 指令介绍，请参考：[PZ-SIM800C GSM-GPRS 模块\SIM800C 模块资料\SIM800 Series_AT CommandManual_V1.09.pdf](#) 这个文档。

发送给模块的指令，如果执行成功，则会返回对应信息和“OK”，如果执行失败/指令无效，则会返回“ERROR”。

另外，提醒大家，所有的指令都必须是以 ASCII 编码字节格式发送，不要在指令里面夹杂中文符号。同时，很多指令都带有查询或提示功能，可以通过指令+“?”来查询当前设置，通过指令+“=?”的方式来获取设置提示。

1.3.3 拨打/接听电话/DTMF 检测

使用 PZ-SIM800C 模块，我们可以很方便的进行电话的拨打与接听。

本节，将要用到的指令有：

ATE1/ATD/ATA/ATH/AT+COLP/AT+CLIP/AT+VTS/AT+DDET 等 8 条 AT 指令。

ATE0，用于关闭回显，在通过电脑串口调试助手调试的时候，我们发送：ATE1，开启回显，可以方便调试，但是我们通过单片机程序控制的时候，用不到回显功能，所以发送：ATE0，将其关闭。

ATD，用于拨打任意电话号码，格式为：ATD+号码+；，末尾的’；’一定要加上，否则不能成功拨号，如发送：ATD10010；，即可实现拨打 10010。

ATA，用于应答电话，当收到来电的时候，给模块发送：ATA，即可接听来电。

ATH，用于挂断电话，要想结束正在进行的通话，只需给模块发送：ATH，即可挂断。

AT+COLP，用于设置被叫号码显示，这里我们通过发送： AT+COLP=1，开启被叫号码显示，当成功拨通的时候（被叫接听电话），模块会返回被叫号码。

AT+CLIP，用于设置来电显示，通过发送： AT+CLIP=1，可以实现设置来电显示功能，模块接收到来电的时候，会返回来电号码。

AT+VTS，产生 DTMF 音，该指令只有在通话进行中才有效，用于向对方发送 DTMF 音，比如在拨打 10010 查询的时候，我们可以通过发送： AT+VTS=1，模拟发送按键 1。

AT+DDET，用于设置 DTMF 解码功能，该指令要在电话连接之前发送才有效，通过发送 AT+DDET=1，开启在通话中进行 DTMF 的检测，比如，在通话中，对方在移动手机设备上按下数字 1 时，这时模块会返回按下的数字 1。

以上就是在拨打/接听电话时经常用到的几条指令，通过这几条指令，就可以实现电话的拨打和接听了，**不过首先要保证模块成功接入到 GSM 网络**，通过发送： AT+COPS?，如果返回： +COPS: 0,0,"CHN-UNICOM"，则说明模块成功连接到了 GSM 网络，可以正常使用了，网络运营商为"CHN-UNICOM"（中国联通）。

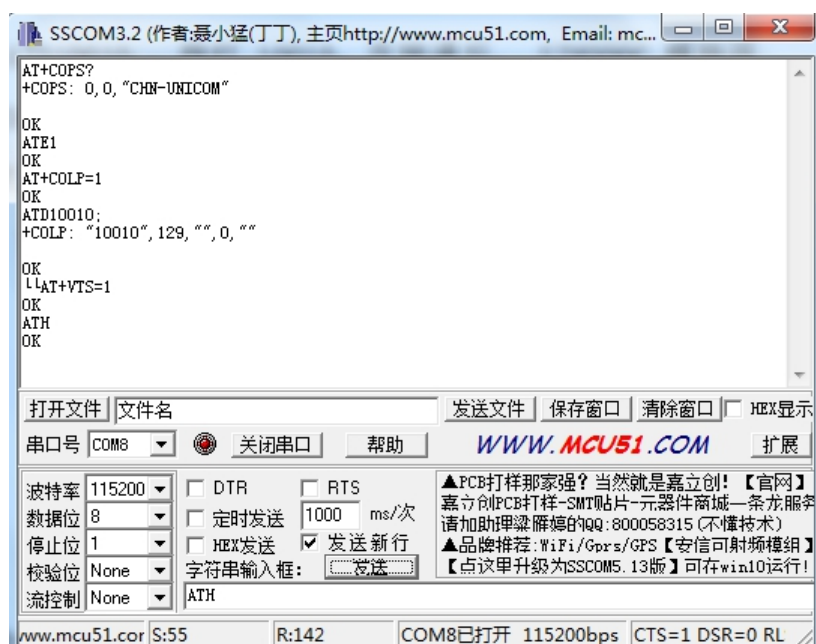
1.3.3.1 拨打电话

本节，我们将利用 PZ-SIM800C 模块来拨打 10010，并进行话费查询。

首先发送： ATE1，设置回显，再发送： AT+COLP=1，设置被叫号码显示。如图所示：



然后，我们发送： ATD10010;，拨打 10010，在接通后， SIM800C 模块返回： +COLP: "10010", 129, "", 0, "", 此时，我们就可以听到中国联通那熟悉的声音了.... 待一堆废话结束后，我们发送： AT+VTS=1，即可查询本机电话号码。最后，通过发送： ATH，挂断，结束本次通话。如图所示：



至此，我们就完成了一次拨号、发送 DTMF 音、结束通话的操作。十分简单，所以用 PZ-SIM800C 模块来实现打电话，那实在是小菜一碟。

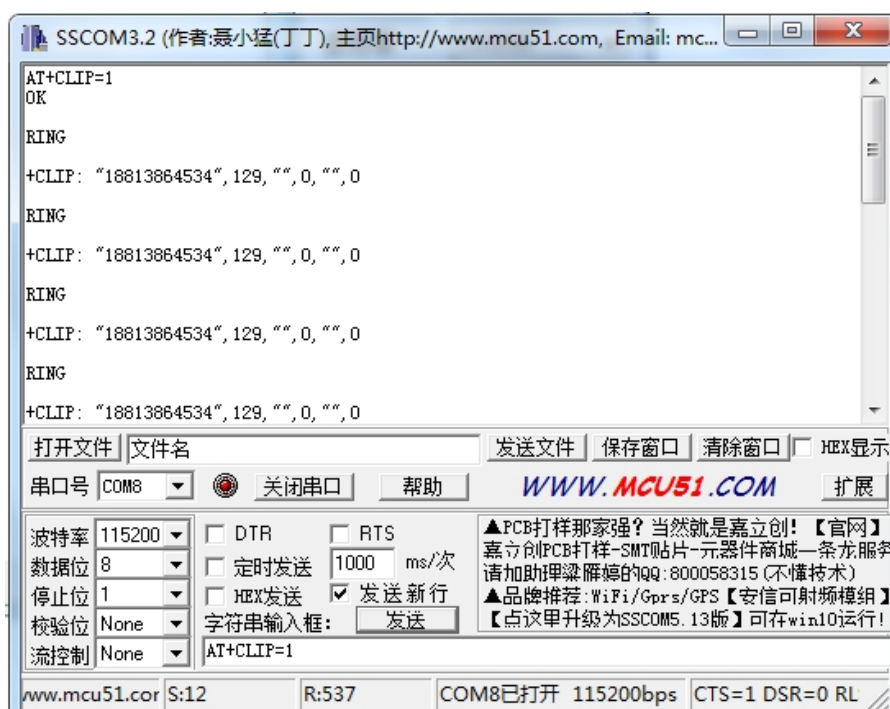
1.3.3.2 接听电话

本节，我们将通过 PZ-SIM800C 模块实现电话接听。

首先，我们发送： AT+CLIP=1，开启来电显示功能，然后用其他电话机/手机拨打模块上 SIM 卡的号码。然后，模块在接收到来电的时候，会通过耳机输出来电铃声，并且可以在串口接收到来电号码，如： +CLIP:

"18813864534", 129, "", 0, "", 0, 0, 表示当前接入号码为： 18813864534。

此时，我们发送： ATA，即可接听来电，并进行通话。当对方挂断电话的时候， SIM800C 模块会返回： NO CARRIER，并结束此次通话如图所示。当然，我们也可以通过发送： ATH，来主动结束通话。



1.3.3.3 DTFM 检测

本节，我们在上述电话接听的基础上，通过 PZ-SIM800C 模块实现 DTMF 检测。

首先，我们确保模块是没有在通话状态下的，这时我们发送：AT+DDET=1，开启 DTMF 检测，如图所示：



然后用其他电话机/手机拨打模块上 SIM 卡的号码。然后，模块在接收

到来电的时候，会通过耳机输出来电铃声，并且可以在串口接收到来电号码，如：`+CLIP: "18813864534",129,"",0,"",0,0`表示当前接入号码为：18813864534。

当接收到来电后，发送：ATA，即可接听来电，当我们在移动手机设备上（或者电话机）按下不同的数字，串口助手将会返回相对应的字符，这就实现了DTMF检测功能。最后可以发送：ATH，来挂断电话。

这些指令的具体使用可以参考“PZ-SIM800C GSM-GPRS 模块\SIM800C 模块资料\SIM800 Series_AT CommandManual_V1.09.pdf”这个文档。

1.3.4 短信的读取与发送

使用 PZ-SIM800C 模块，我们可以很方便的进行中英文短信的读取与发送。短信的读取与发送将用到的指令有：AT+CNMI / AT+CMGF / AT+CSCS / AT+CSMP / AT+CMGR/AT+CMGS/AT+CPMS 等 7 条 AT 指令。

AT+CNMI，用于设置新消息指示。发送：AT+CNMI=2,1，设置新消息提示，当收到新消息，且 SIM 卡未满的时候，SIM800C 模块会通过串口输出数据，如：`+CMTI: "SM",2`，表示收到接收到新消息，存储在 SIM 卡的位置 2。

AT+CMGF，用于设置短消息模式，SIM800C 支持 PDU 模式和文本（TEXT）模式等 2 种模式，发送：AT+CMGF=1，即可设置为文本模式。

AT+CSCS，用于设置 TE 字符集，默认的为 IRA，国际标准字符集，在发送纯英文短信的时候，发送：AT+CSCS="GSM"，设置为缺省字符集即可。在发送中英文短信的时候，需要发送：

AT+CSCS="UCS2"，设置为 16 位通用 8 字节倍数编码字符集。

AT+CSMP，用于设置短消息文本模式参数，在使用 UCS2 方式发送中文短信的时候，需要发送：AT+CSMP=17,167,2,25，设置文本模式参数。

AT+CMGR，用于读取短信，比如发送：AT+CMGR=1，则可以读取 SIM 卡存储在位置 1 的短信。

AT+CMGS，用于发送短信，在"GSM"字符集下，最大可以发送 180 个字节的英文字符，在"UCS2"字符集下，最大可以发送 70 个汉字（包括字符/数字）。

AT+CPMS，用于查询/设置优选消息存储器，通过发送：AT+CPMS?，可以查询

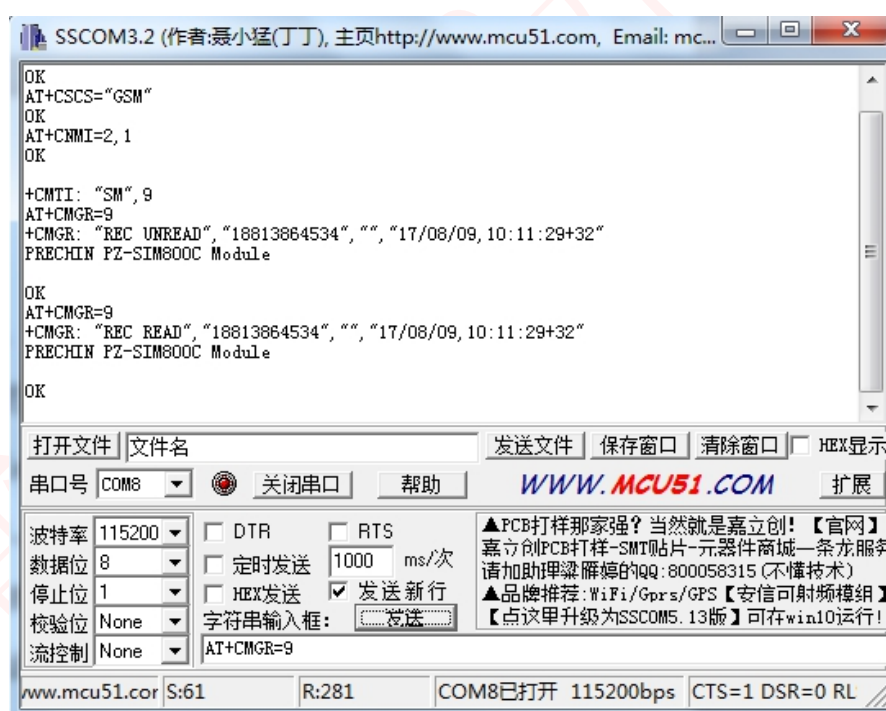
当前 SIM 卡最大支持多少条短信存储，以及当前存储了多少条短信等信息。如返回：+CPMS:"SM_P",1,50,"SM_P",1,50,"SM_P",1,50,表示当前 SIM 卡最大存储 50 条信息，目前已经有 1 条存储的信息。

1.3.4.1 英文短信的读取

本节，我们将用其他手机先发送一条英文短息到 PZ-SIM800C 模块上，然后读取模块接收到的这条英文短信。

首先，我们发送：AT+CMGF=1，设置为文本模式，然后发送：AT+CSCS="GSM"，设置 GSM 字符集，然后发送：AT+CNMI=2,1,设置新消息提示。接着，我们用别的手机发送一条英文短信“PRECHIN PZ-SIM800C Module”到我们的模块上（如果不知道模块号码，可以发送：AT+CNUM，查询模块号码）。

模块接收到短信后，会提示如：+CMTI:"SM",9,表明收到了新的短信，存放在 SIM 卡位置 9。然后，我们发送 AT+CMGR=9，即可读取该短信，如图所示：



从上图可以看到，第一次发送 AT+CMGR=9 的时候，SIM800C 模块返回的是（省略了多余的回车换行和“OK”等字符串，下同）：

```
+CMGR: "REC UNREAD", "18813864534", "", "17/08/09,10:11:29+32"
PRECHIN PZ-SIM800C Module
```

其中：“REC UNREAD”，表示该短信没有被读取过，也就

是未读短信。接着是：“18813864534”，表示此短信发送方的电话号码。最后的：“17/08/09,10:11:29+32”，表示的是此短信的接收日期和时间信息。然后换行后的“PRECHIN PZ-SIM800C Module”，则表示读取到的短信内容，这与我们发送的内容一致，说明实验成功。图中，我们发送了 2 次 AT+CMGR=9，读取了 2 次，可以看到第一次读取的时候，短信为“REC UNREAD”，第二次的时候，短信状态变为了“REC READ”，表示此短信已经被读取过了。

1.3.4.2 英文短信的发送

本节，我们将利用 PZ-SIM800C 模块来给指定手机号码发送一条英文短信。

这里，我们用到 AT+CMGS 指令来发送短信，发送全英文/数字短信的时候，我们先设置为：“GSM”字符集（AT+CSCS=“GSM”），文本模式（AT+CMGF=1）。

假设我们要给手机号：18813864534 发送一条短信，则发送：

AT+CMGS="18813864534", 然后模块返回：>，此时我们输入我们需要发送的内容：PZ-SIM800C MSG SEND TEST，注意，此可以不用发送回车了。在发送完内容以后，最后以十六进制（HEX）格式单独发送（不用添加回车）：1A（即 0X1A，即“CTRL+Z”的键值，用于告诉 SIM800C，要执行发送操作。另外还可以发送：0X1B，即“ESC”的键值，用于告诉 SIM800C，取消本次操作，不执行发送。），即可启动一次短信发送。

稍等片刻，在短信成功发送后，模块返回如：+CMGS: 18，的确认信息，表示短信成功发送，其中 18 为模块内部的短信计数器，一般不用理会。如图所示：



从上图可以看出，我们在短信发送的最后，输入的是十六进制的 1A，将这个数据发送给 SIM800C 模块后，启动短信发送，然后发送成功后，模块返回了：+CMGS: 18，说明短信发送成功了。

1.3.4.3 中英文短信的读取

本节，我们将用其他手机先发送一条中英文短信到 PZ-SIM800C 模块上，然后读取模块接收到的这条中英文短信。

中文短信的读取，我们这里采用与英文短信读取一样的方式（文本模式，GSM 字符集），读取方法也是一样，不过由于是中文短信，返回的数据同全英文/数字短信有所区别。全英文/数字短信，返回的短信内容，我们可以直接读取。但是中英文短信，返回内容却是其 UNICODE 编码的组合（即所有汉字/字符都采用 UNICODE 编码）。所以，我们还需要将 UNICODE 编码的内容，转换为汉字/字符。

首先，我们发送：AT+CMGF=1，设置为文本模式，然后发送：AT+CSCS="GSM"，设置 GSM 字符集，然后发送：AT+CNMI=2,1，设置新消息提示。接着，我们用别的手机发送一条中英文短信“PZ-SIM800C 中英文短信测试”到我们的模块上。

模块接收到短信后，会提示如：+CMTI: "SM",11，表明收到了新的短信，存放在 SIM 卡位置 11。然后，我们发送 AT+CMGR=11，即可读取该短信，如图所示：



读取短信后，收到的内容如下：

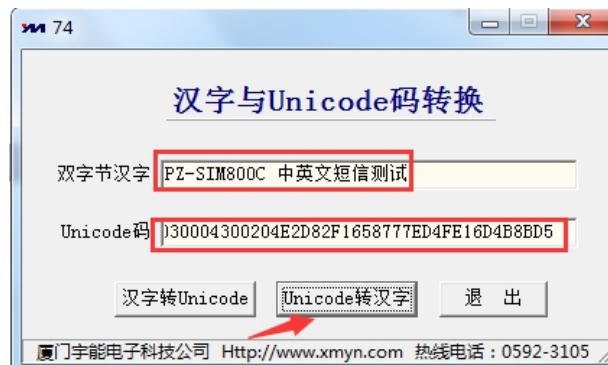
+CMGR: "REC UNREAD","18813864534","","17/08/09,10:32:29+32"

0050005A002D00530049004D003800300030004300204E2D82F1658777ED4FE16D4B8BD5

可以看到，和英文短信的读取基本一样，只是返回的短信内容，是一串 UNICODE 编码，所以我们这里需要用到一个工具：汉字 UNICODE 互换工具.exe，来实现 UNICODE 到汉字的装换。该软件在“PZ-SIM800C GSM-GPRS 模块\调试工具\汉字 Unicode 互换工具”下可以找到。

打开该软件，然后将收到的 UNICODE 编码内容：

0050005A002D00530049004D003800300030004300204E2D82F1658777ED4FE16D4B8BD5（这里是 UNICODE 编码的字符串，每 4 个字符组成 1 个 UNICODE 码，比如 0041，实际上是代表十六进制的 0X0041，即大写字母‘A’的 UNICODE 码。中英文短信都是（不论读短信还是发短信）采用 UNICODE 编码的字符串，所以字符串长度必定为 4 的倍数），复制到 Unicode 码输入栏，然后点击“Unicode 转汉字”按钮，即可在双字节汉字一栏，看到转换过来的短信内容，如图所示：



可以看到，转换后的内容为：PZ-SIM800C 中英文短信测试。这就是我们发送给模块的内容，说明中英文短信读取成功。PS：如果你转换后有几个问号，请在 Unicode 字符串最后加入一个或多个空格就好了，这是软件的 bug。

1.3.4.4 中英文短信的发送

本节，我们将利用 PZ-SIM800C 模块来给指定手机号码发送一条中英文短信。

同样，我们用到 AT+CMGS 指令来发送中英文短信。还是采用文本模式发送，发送步骤如下：

首先，发送：AT+CMGF=1， 设置为文本模式。然后，发送 AT+CSMP=17,167,2,25，设置文本模式参数。最后发送： AT+CSCS=" UCS2"，设置为 UCS2 编码字符集。

此时，我们便可以发送 AT+CMGS 指令来发送中英文短信了，不过由于使用了 UCS2 字符集，所有字符/数字/汉字，都必须使用 UNICODE 编码。

假定我们要给手机号：18813864534，发送一条中英文短信，内容为“PZ-SIM800C 中英文短信发送测试”。我们首先要用：汉字 UNICODE 互换工具.exe，这个软件将号码和发送内容转换为 UNICODE 字符串（需要手动去掉空格），得到：

18813864534，转换后的 UNICODE 字符串（去掉空格后）为：

00310038003800310033003800360034003500330034

PZ-SIM800C 中英文短信发送测试，转换后的 UNICODE 字符串为：

0050005A002D00530049004D003800300030004300204E2D82F1658777ED4FE15
3D190016D4B8BD5

然后，发送： AT+CMGS=

00310038003800310033003800360034003500330034” ,这个指令, 然后模块返回:>, 此时我们输入我们需要发送的内容:

0050005A002D00530049004D003800300030004300204E2D82F1658777ED4FE153D190016D4B8BD5, 注意, 此可以不用发送回车了。在发送完内容以后, 最后以十六进制 (HEX) 格式单独发送 (不用添加回车): 1A (即 0X1A), 启动一次短信发送, 如图所示:



等到模块发送完成, 返回如: +CMGS: 19, 表示短信已成功发送。此时, 我们在目标手机 (18813864534) 上面, 就可以看到模块发送过来的中英文短信了。

短信的读取与发送, 我们就介绍到这里, 我们全部是使用文本 (TEXT) 模式来实现的, 当然还可以用 PDU 模式, 不过 PDU 模式稍微复杂点, 有兴趣的朋友, 可以参考相关资料自行研究下。

1.3.4.5 彩信的发送与读取

关于彩信发送与读取的使用说明请查看 PZ-SIM800C 彩信功能使用. pdf, 文件路径: PZ-SIM800C GSM-GPRS 模块\ PZ-SIM800C 彩信功能使用. pdf。

以上就是短信读取与发送需要用到的一些 AT 指令, 这些指令的使用可以参考 “ PZ-SIM800C GSM-GPRS 模块 \SIM800C 模块 资料 \SIM800 Series_AT CommandManual_V1.09. pdf ” 这个文档。

1.3.5 GPRS 通信

PZ-SIM800C 模块内嵌了 TCP/IP 协议，通过该模块，我们可以很方便的进行 GPRS 数据通信。本文档例程我们将实现模块与电脑的 TCP 和 UDP 数据传输。将要用到的指令有：

AT+CGCLASS/AT+CGDCONT/AT+CGATT/AT+CIPCSGP/AT+CIPHEAD/AT+CLPORT/AT+CIPSTART/AT+CIPSEN/AT+CIPSTATUS/AT+CIPCLOSE/AT+CIPSHUT 等 11 条 AT 指令。

AT+CGCLASS，用于设置移动台类别。SIM800C 模块支持类别“B”、“CG”和“CC”，发送：AT+CGCLASS=“B”，设置移动台类别为 B。即，模块支持包交换和电路交换模式，但不能同时支持。

AT+CGDCONT，用于设置 PDP 上下文。发送：AT+CGDCONT=1,“IP”,“CMNET”，设置 PDP 上下文标志为 1，采用互联网协议（IP），接入点为“CMNET”。

AT+CGATT，用于设置附着和分离 GPRS 业务。发送：AT+CGATT=1，附着 GPRS 业务。

AT+CIPCSGP，用于设置 CSD 或 GPRS 链接模式。发送：AT+CIPCSGP=1,“CMNET”，设置为 GPRS 连接，接入点为“CMNET”。

AT+CIPHEAD，用于设置接收数据是否显示 IP 头。发送：AT+CIPHEAD=1，即设置显示 IP 头，在收到 TCP/UDP 数据的时候，会在数据之前添加如：
+IPD:28，表示是 TCP/UDP 数据，数据长度为 28 字节。通过这个头，可以方便我们在程序上区分数据来源。

AT+CLPORT，用于设置本地端口号。发送：AT+CLPORT=“TCP”,“8888”，即设置 TCP 连接本地端口号为 8888。

AT+CIPSTART，用于建立 TCP 连接或注册 UDP 端口号。发送：AT+CIPSTART=“TCP”,“219.137.88.114”,“8086”，模块将建立一个 TCP 连接，连接目标地址为：219.137.88.114，连接端口为 8086，连接成功会返回：CONNECT OK。

AT+CIPSEND，用于发送数据。在连接成功以后发送：AT+CIPSEND，模块返回：>，此时可以输入要发送的数据，最大可以一次发送 1352 字节，数据输入完后，同发短信一样，输入十六进制的：1A（0X1A），启动发送数据。在数据发送完成后，模块返回：SEND OK，表示发送成功。

AT+CIPSTATUS，用于查询当前连接状态。发送： AT+CIPSTATUS，模块即返回当前连接状态。

AT+CIPCLOSE，用于关闭 TCP/UDP 连接。发送： AT+CIPCLOSE=1，即可快速关闭当前 TCP/UDP 连接。

AT+CIPSHUT，用于关闭移动场景。发送： AT+SHUT，则可以关闭移动场景，关闭场景后连接状态为： IP INITIAL，可以通过发送： AT+CIPSTATUS，查询。另外，在连接建立后，如果收到： +PDP: DEACT，则必须发送： AT+CIPSHUT，关闭场景后，才能实现重连。

以上就是 GPRS 通信（TCP/UDP）将要用到的一些 AT 指令的简介，这些指令的具体使用可以参考“PZ-SIM800C GSM-GPRS 模块\SIM800C 模块资料\SIM800 Series_AT CommandManual_V1.09.pdf”这个文档。

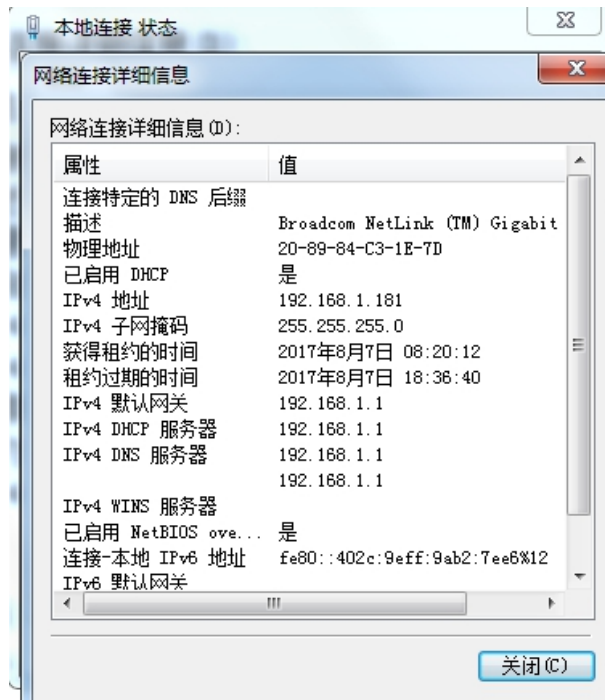
另外，要实现模块与电脑的 GPRS 通信，**需要确保所用电脑具有公网 IP，否则无法实现通信，推荐在 ADSL 网络下进行测试，并最好关闭防火墙/杀毒软件。**

对于 ADSL 用户（没用路由器），直接拥有 1 个公网 IP，你可以通过百度，搜索： IP，第一个条目，就是本机 IP，如图所示：



该 IP 将与你的电脑 IP（双击本地连接图标→支持选项卡，即可查看）是一致的。

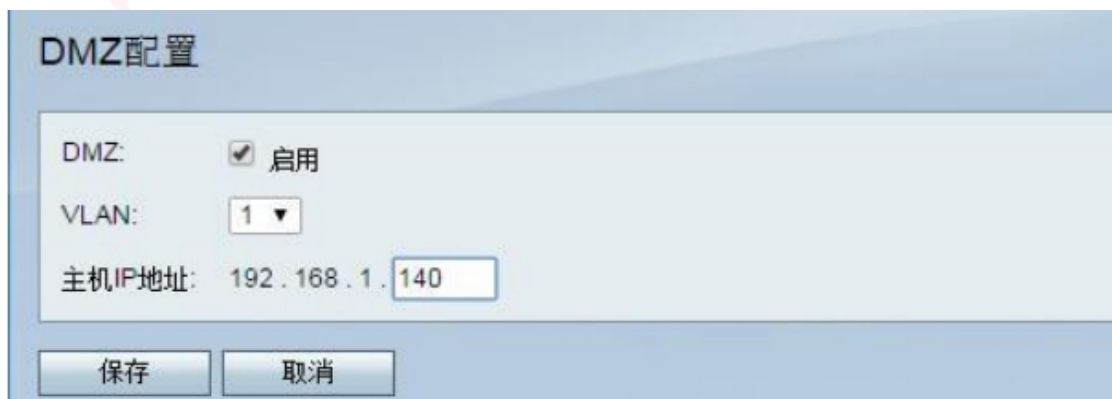
对与使用了路由器的 ADSL 用户，那么电脑 IP 与你百度到的公网 IP 是不一样的，如图所示：



可以看到，我们电脑 IP 为 192.168.1.181，与公网 IP 不一致，此时我们需要对路由器进行一下转发规则设置：登录路由器控制页面，然后选择→LAN 接口配置→DMZ 配置，如图所示：



然后设置启用 DMZ 主机，并设置 DMZ 主机 IP 地址为所用电脑的 IP 地址，本机 IP 为：192.168.1.181，如图所示：



然后保存。这样我们就把内网 IP (192.168.1.181) 映射到了外网，相当于经过路由器的电脑，拥有了一个公网 IP。

最后，我们在电脑上，还需要用到一个软件：网络调试助手，来协助验证 GPRS 通信，该软件启动界面如图所示：



该软件的使用非常简单，我们将在第四节配合我们的例程向大家介绍该软件的使用。

1.3.6 蓝牙通信

PZ-SIM800C 模块集成了蓝牙 3.0，通过该模块，我们可以很方便的进行蓝牙数据通信。本文档例程我们将实现模块与手机蓝牙数据传输。将要用到的指令有：

AT+BTPOWER/AT+BTHOST/AT+BTSCAN/AT+BTUNPAIR/AT+BTPAIR/AT+BTACPT/AT+BTSPPSSEND/AT+BTDISCONN 等 8 条 AT 指令。

AT+BTPOWER, 用于设置开启或关闭蓝牙电源，当发送 AT+BTPOWER=1，返回

OK, 表示开启蓝牙电源; 发送 AT+BTPOWER=0, 返回 OK, 表示关闭蓝牙电源。

AT+BTHOST, 用于查询和设置当前模块蓝牙设备名, 当发送 AT+BTHOST? 时, 返回该设备的蓝牙名字和地址, 设置当前模块蓝牙设备名时, 命令格式为 AT+BTHOST=<name>, name 为你要设置的设备名。

AT+BTSCAN, 用于设置蓝牙搜索参数, 发送 AT+BTSCAN=1, 10, 开启扫描设备, 时间为 10s, 搜索到设备返回例如: +BTSCAN: 0, 1, "Meizu MX4 Pro", 22:22:5f:b8:e9:af, -79, 表示设备 1, 名称: Meizu MX4 Pro, 地址: 22:22:5f:b8:e9:af, 信号: -79。

AT+BTUNPAIR 用于删除蓝牙设备配对信息, 发送 AT+BTUNPAIR=0, 删除所有已配对的蓝牙设备信息。(**注意: 上次配对过的设备, 下次进行配对前必须删除配对信息**)

AT+BTPAIR 用于实现蓝牙配对, 发送: AT+BTPAIR=0, 1, 向设备 1 发起配对请求。

AT+BTACPT 用于接收配对的蓝牙设备的连接请求, 发送 AT+BTACPT=1, 接收连接请求, 发送 AT+BTACPT=0, 拒绝连接请求。

AT+BTSPSEND 用于蓝牙串口发送数据, 发送数据有两种方式, 定长与非定长。在连接成功以后发送: AT+CIPSEND, 模块返回: >, 即非定长模式下, 此时可以输入要发送的数据, 最大可以一次发送 1024 字节, 数据输入完后, 同发短信一样, 输入十六进制的: 1A (0X1A), 启动发送数据。在数据发送完成后, 模块返回: SEND OK, 表示发送成功。关于定长模式, 请参考文档《 PZ-SIM800C 蓝牙功能_AN1708C. pdf 》, 文件路径: PZ-SIM800C GSM-GPRS 模块\PZ-SIM800C 蓝牙功能_AN1708C. pdf。

AT+BTDISCONN 用于断开已连接的蓝牙设备服务, 发送 AT+BTDISCONN=1, 断开与设备 1 服务的连接。

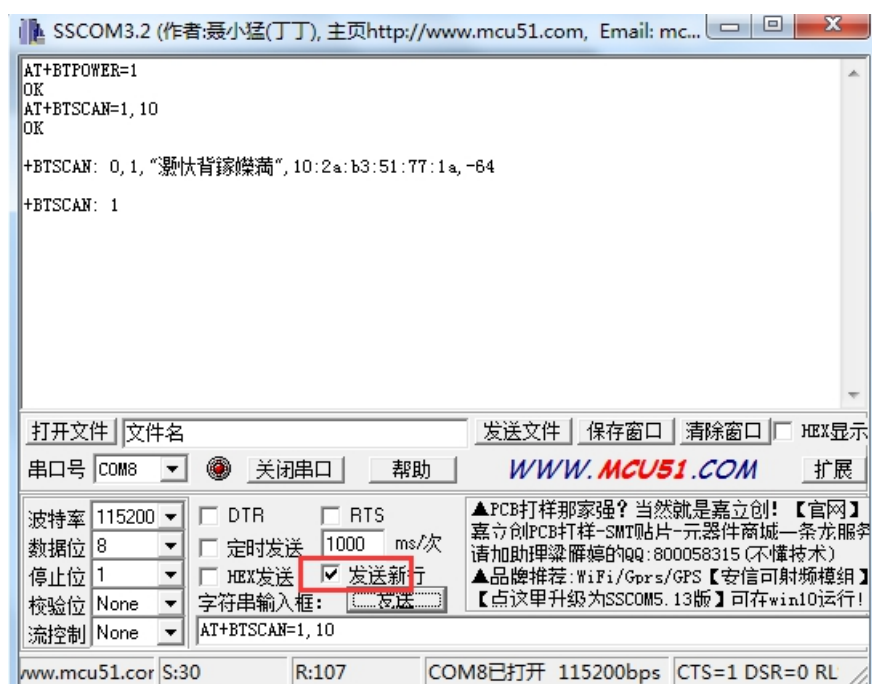
以上就是蓝牙通信将要用到的一些 AT 指令的简介, 这些指令的使用示例可以参考《PZ-SIM800C 蓝牙功能_AN1708C. pdf》或《SIM800 Series_AT Command Manual_V1.09. pdf》。

1.3.6.1 蓝牙 SPP 通信

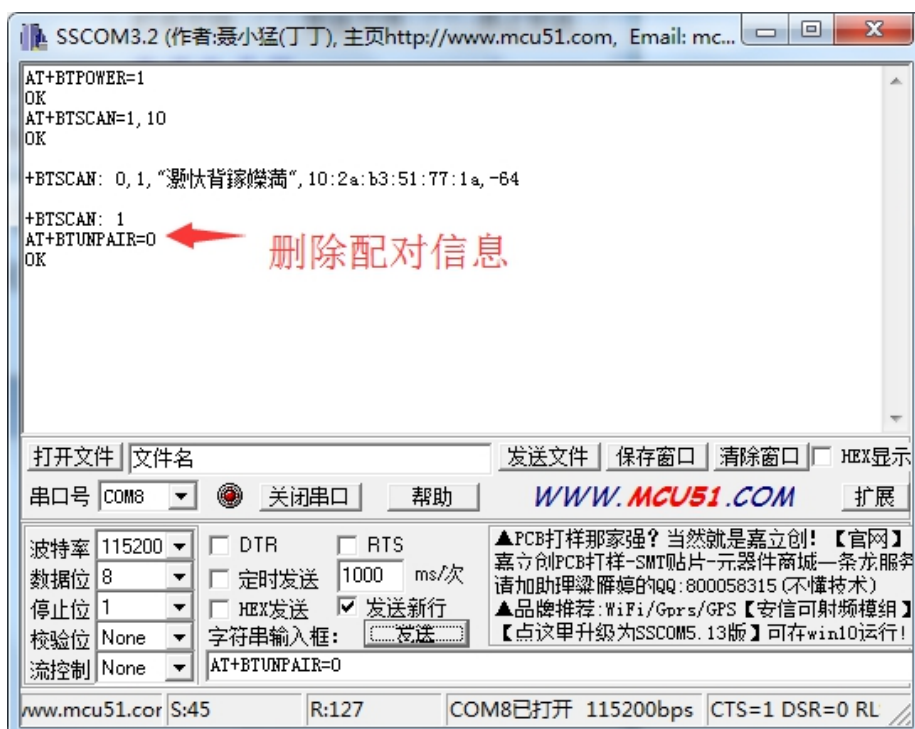
本节, 我们将 PZ-SIM800C 模块与手机进行蓝牙通信。手机端需要安装一个

软件：蓝牙串口助手增强版_Bluetooth_spp_pro（蓝牙串口助手 V0.16.apk），该软件你可以在豌豆荚市场搜索到，也可以在我们提供的资料中找到“PZ-SIM800C GSM-GPRS 模块\调试工具”。

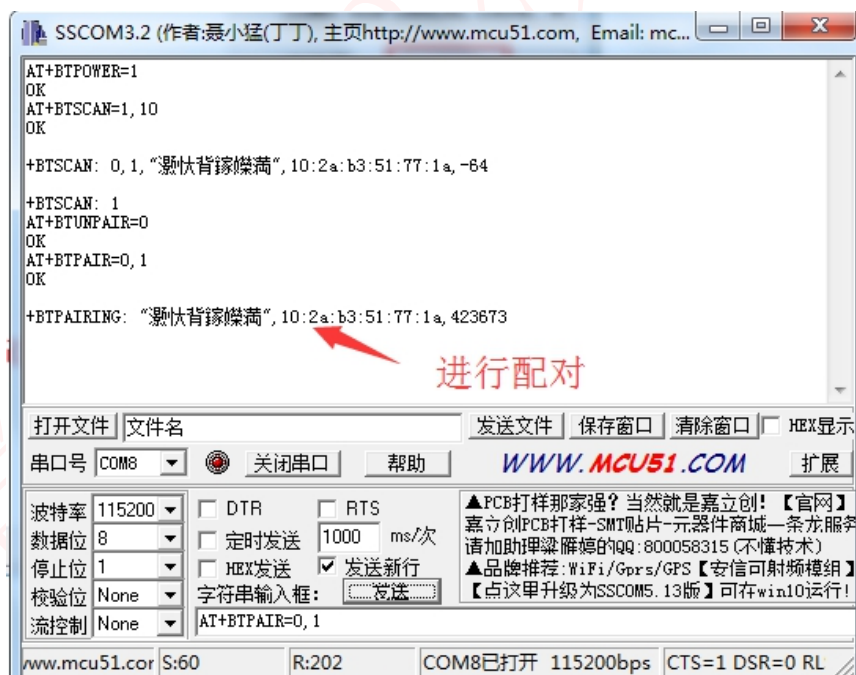
首先，我们发送 AT+BTPOWER=1，打开蓝牙电源，然后发送 AT+BTSCAN=1, 10，启动扫描周边设备，时间为 10S，（注意：必须等待到 +BTSCAN:1 信息返回才能继续发送指令，不然发送指令失败）如图所示：



由于之前 PZ-SIM800C 模块与当前搜索到的手机设备进行过配对连接，保存有配对信息，当模块再次发起配对连接（设备 1）是会出现错误，所以必须发送 AT+BTUNPAIR=0，删除配对信息。如图所示：

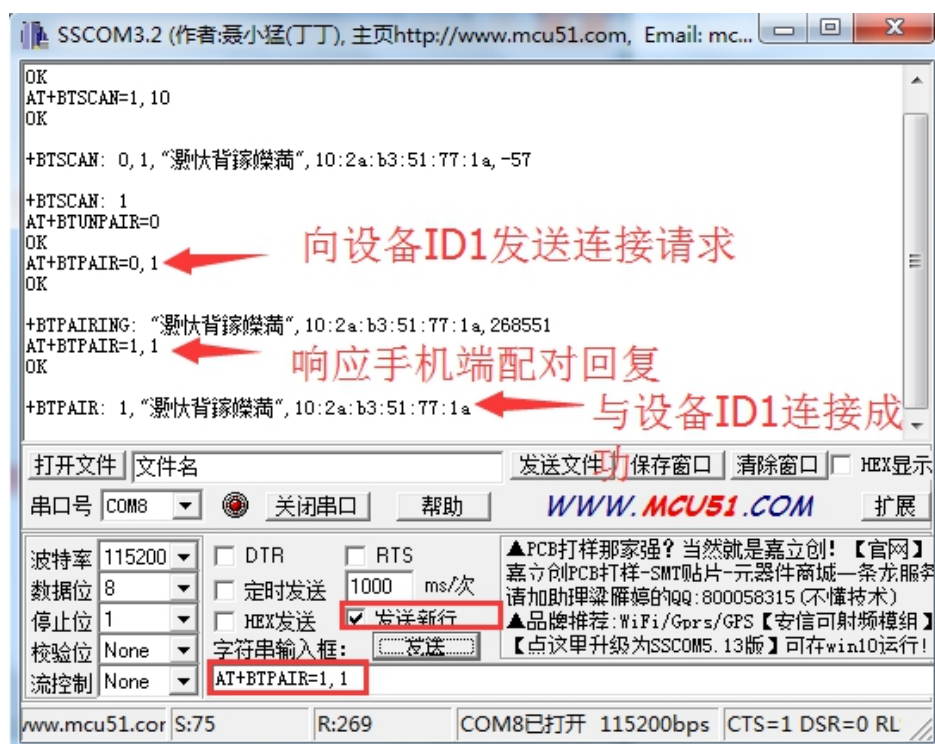


删除完配对信息后，再次发送 AT+BTPAIR=0, 1; 向设备 1 发起配对请求，如图所示：



这时手机端接收到蓝牙配对请求，手机端按下配对回复，这时 PZ-SIM800C 发送指令 AT+BTPAIR=1, 1; 响应手机端的配对回复，当返回+BTPAIR: 1, "灏伙背稼燦满", 10:2a:b3:51:77:1a, 说明与设备 1 已配对成功（图中出现乱码是由于手机端蓝牙名称字符等问题，不用管它），如图所示模块连接成功与所示手机连

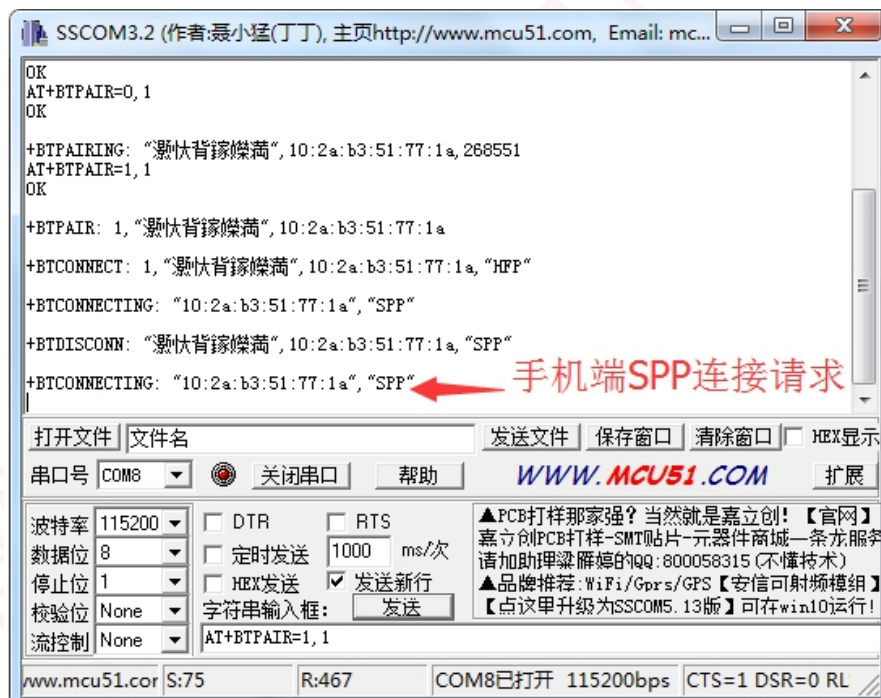
接成功:



PZ-SIM800C 虽然与手机端连接了，但是是还不能进行 SPP 数据传输的，这时手机端打开刚刚安装的蓝牙串口助手增强版 app（蓝牙串口助手 V0.16.apk），点击扫描，当扫描到 SIM800C 设备时，按照步骤①②进行点击连接，如图所示：

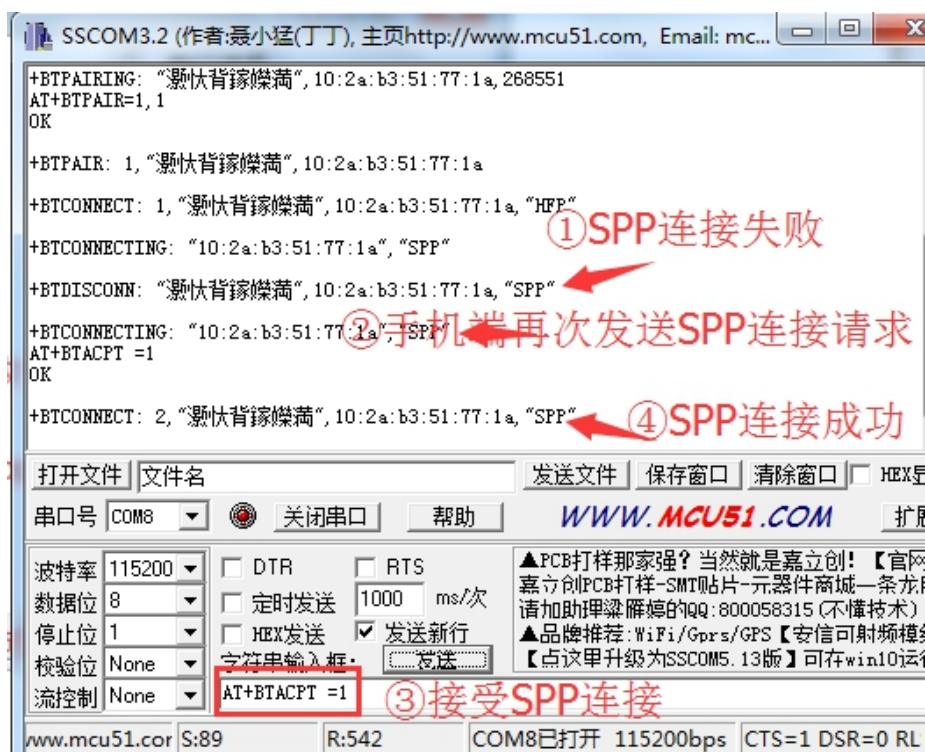


当按下连接设备时在 PZ-SIM800C 模块返回一条手机端 SPP 的连接请求信息如图所示：

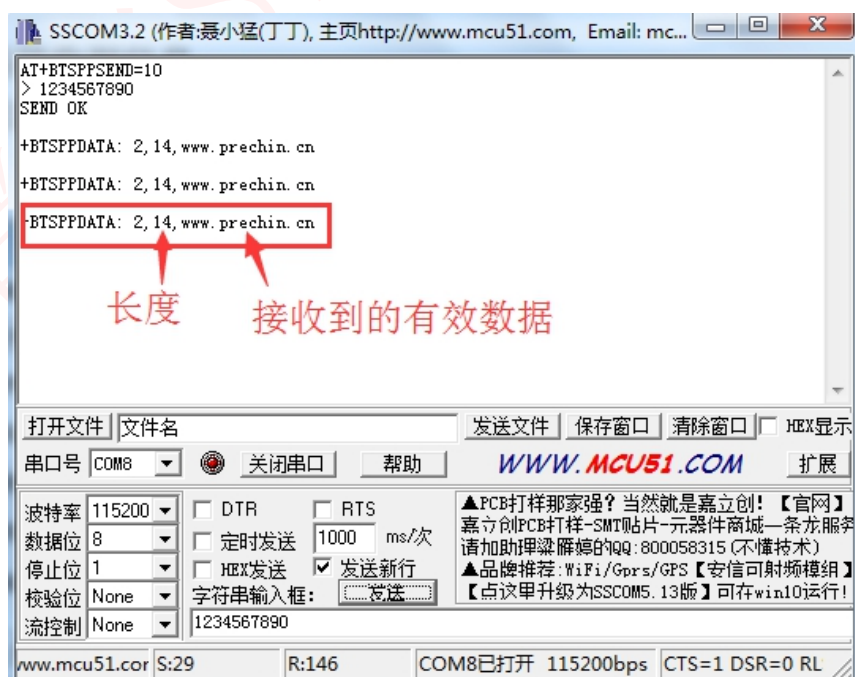


若在一段时间内没有对 SPP 连接请求进行应答，PZ-SIM800C 会返回 +BTDISCONN: “灏怏背稼蝶满”, 10:2a:b3:51:77:1a, “SPP”信息，表示请求连接超时，SPP 连接失败，如图①所示，当出现这种情况时再次按照上面的步骤，手机端点击“连接设备”，再次发送 SPP 连接请求，当 PZ-SIM800C 接收到 SPP 请求时发送指令 AT+BTACPT =1，接收配对蓝牙设备请求，如图③所

示，这时返回 +BTCONNECTING:"10:2a:b3:51:77:1a","SPP"，表示 SPP 已连接成功了，如图④所示：



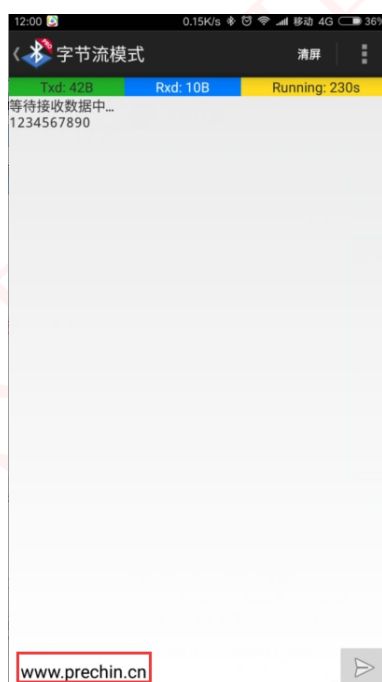
SPP 连接完成后，PZ-SIM800C 发送数据到手机上，先发送指令 AT+BTSPSEND=10 (注意：这里仅以定长方式模式发送数据)，表示接下来要发送 10 个字符，等 > 符号出现时，输入你所要发送的 10 个字节数据，例如：1234567890，然后按发送，这时显示 SEND OK 表示发送数据成功，如图所示：

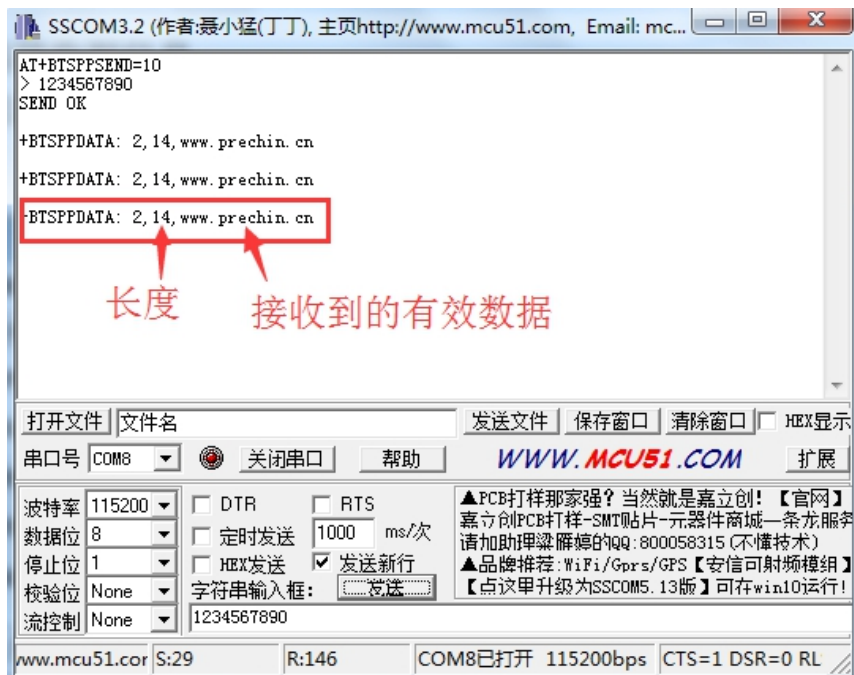


手机端接收到信息 如图所示

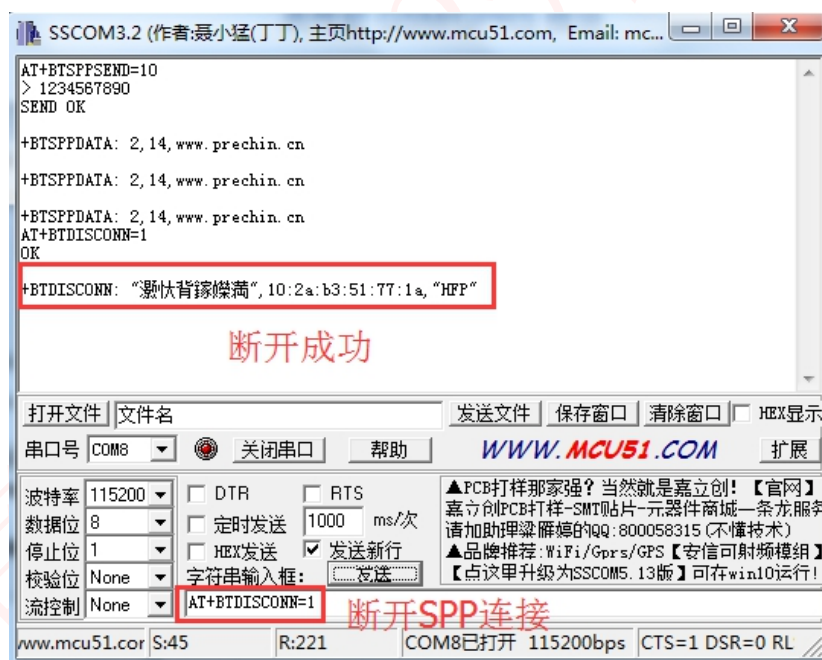


手机端发送数据 www.prechin.cn, PZ-SIM800C 模块接收到手机端发送过来的数据并且自动打印出串口显示, 如图所示:





最后，我们要断开 SPP 连接，发送：AT+BTDISCONN=1，断开与设备 1 的连接如图所示：



若想删除与设备的配对信息，则发送 AT+BTUNPAIR=1，表示删除与当前设备 1 的配对的信息，然后再发送 AT+BTPOWER=0，关闭 PZ-SIM800C 蓝牙电源，即可断开与手机设备的蓝牙连接了。（注意：手机端的蓝牙设置中，在已配对设备栏中会显示上次连接的蓝牙设备，是因为手机端保存了曾经与 PZ-SIM800C 的配对连接信息，所以不需要理会！）

对于以上蓝牙通信的配置操作步骤有不理解的地方和具体需要详细的使用过程，请参考文档 PZ-SIM800C 蓝牙功能使用.pdf。

1.3.7 TTS 文本转语音

关于 TTS 文本转语音的使用说明请查看《PZ-SIM800C TTS 功能_AN1708.pdf》，文件路径：PZ-SIM800C GSM-GPRS 模块\PZ-SIM800C TTS 功能_AN1708.pdf，在这里我们就不做出介绍了。

1.3.8 录音

关于录音的使用说明请查看 PZ-SIM800C 录音功能使用.pdf。

1.3.9 模块使用注意事项总结

(1) 测试前，请确保供电电源功率足够，外部电源输入范围是 5-24V，功率一般要求大于 10W，也就是 5V 需要有 2A，12V 需要 1A 的电源。

(2) 请确保模块已经正常启动，自动启动的方法是用跳线帽短接 P1 功能选择接口的 PKEY 与 VBAT，手动启动的方法是长按 KEY1 按键 1-3 秒，直到红灯开始闪烁。

(3) 请确保模块的跳线帽正常连接，如果是用 USB 串口测试，模块的跳线帽 SRXD 连接 RTXD, STXD 连接 RRXD。

(4) 使用串口助手调试，发送 AT 命令要选择“发送新行”，请仔细看我们前面使用的截图。

2 硬件设计

本实验功能简介：本实验用于测试 PZ-SIM800C GSM/GPRS 模块，总共包括四大项测试：

1，拨号测试—通过按 KEY0 按键进入此项测试。进入测试后，屏幕将虚拟一个键盘，通过键盘输入电话号码，即可进行拨号。如果有电话打进来，则会显

示来电号码，并可以通过键盘实现来电接听。

2，短信测试—通过按 KEY1 按键进入此项测试。此项测试包含 2 个子项：读短信测试和发短信测试。按 KEY0 进入读短信测试，屏幕将显示 SIM 卡当前存储的信息条数以及总共可以存储的信息条数，并在屏幕上虚拟一个键盘，通过键盘输入，即可读取指定条目的短信，并且语音报读，其内容将显示在 LCD 上面。按 KEY1 进入发短信测试，屏幕将显示一条固定的短信内容，并虚拟一个键盘，通过键盘输入目标手机号码，即可执行发送，将固定内容的短信发送给目标手机，并带状态提示。

3，GPRS 测试—通过按 KEY_UP 按键进入此项测试。此项测试又包含 2 个子项：TCP 测试和 UDP 测试。默认为 TCP 连接，通过按 KEY_UP 按键，可以在 TCP/UDP 之间切换。此项测试需要输入 IP 地址（要连接的目标 IP 地址，必须为公网 IP），端口号固定为：8086。在设定好连接方式和 IP 地址之后，即可进行连接，连接成功后，则可以和目标进行 GPRS 数据通信。本测试，我们在电脑和 PZ-SIM800C 模块之间实验，电脑端需要一个软件：网络调试助手，来实现和模块的 TCP/UDP 数据通信测试。

4，蓝牙测试—通过先按 KEY2 按键（**天马&麒麟开发板有此按键，对于 F407 最小系统可选择屏蔽一个其他功能，挪用一個按键出来**），然后进入此项测试。此项测试又包含 2 个子项：发起配对请求和接收配对请求模式的通信，按 KEY2 进入发起配对请求，然后通过扫描搜索到手机设备，建立连接后，手机端打开蓝牙调试助手与模块再一次进行 spp 连接，然后手机看到模块发送的数据，屏幕也显示手机端发送过来的数据。按 KEY0 进入接收配对请求模式，手机端连接搜索到模块设备，然后进行连接，spp 的连接和数据通信和前面的效果一样。

2.1 硬件准备

本实验使用到硬件资源如下：

- (1) F407 最小系统&天马&麒麟开发板一块
- (2) PZ-SIM800C GSM/GPRS 模块一个
- (3) 直流稳压电源 1 个（推荐 12V 1A 电源）
- (4) 中国移动/联通 GSM SIM 卡一张（未停机，开通 GPRS 业务）

- (5) 耳机一副（带麦克风功能，用于通话测试）
- (6) 一台支持蓝牙的手机设备（安卓系统）
- (7) TFTLCD 触摸屏一个（对于没有触摸屏的可以选择用串口 AT 指令测试）

要完成本文档例程的所有功能测试，请大家务必准备好以上硬件，否则有些功能可能无法完成。

2.2 模块与开发板连接

PZ-SIM800C 所有的控制与数据，都是通过串口来传输的，所以我们的开发板与模块连接，只需要连接串口即可（当然还需要共地）。

这里以麒麟为例介绍如何连接，接下来，我们看看开发板与 PZ-SIM800C 模块的连接方式，本例程通过开发板的串口 3 连接 PZ-SIM800C 模块，连接方式如下：

1，通过杜邦线连接

这种方式通过杜邦线连接，需要将 PZ-SIM800C 模块 P1 的两个跳线帽拔了（SRXD、STXD 短接的两个）。然后，用 3 根杜邦线，按如下所示关系与麒麟开发板连接：

PZ-SIM800C GSM/GPRS模块	STM32 IO口
GND	GND
SRXD	PB10 (TXD)
STXD	PB11 (RXD)

注意，图中的 GND，大家可以在开发板和 PZ-SIM800C 模块上面，随便找一个 GND 标号的排针，连接在一起即可。

2，通过 USB 转 RS232 线连接

这种方式需要一根两头均为公头的 RS232 直通线，因为天马&麒麟开发板板载了 RS232 模块，所以直接可通过 RS232 对接，**对于 F407 最小系统只能使用方式 1 连接。**

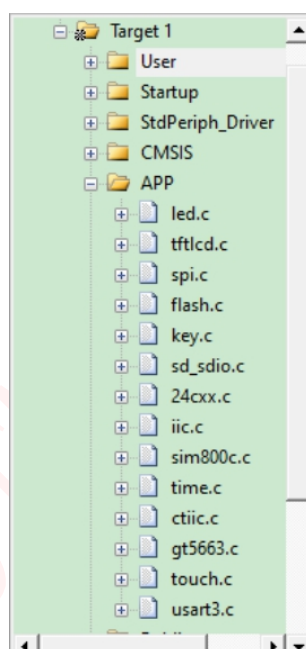
如果没有 RS232 公头线我们推荐采用第 1 种方法来连接开发板和 PZ-SIM800C 模块，最后，特别提醒：PZ-SIM800C 模块必须由单独的电源供电（推荐 12V1A 电源），开发板则可以通过 USB 插电脑供电，不过切记要共地哦！！

3 软件设计

本实验，在开发板的 FLASH 字库实验基础上进行修改，在 APP 文件夹里面新建 usart3 文件夹，存放 usart3.c 和 usart3.h 两个文件。并在工程 APP 组里面添加 usart3.c，并添加 usart3 文件夹到头文件包含路径。

在工程目录 APP 中添加 sim800c 文件夹，新建 sim800c.c 和 sim800c.h 两个文件，存放在 sim800c 文件夹内，将 sim800c.c 加入 APP 工程组，并添加 sim800c 文件夹到头文件包含路径。

我们去掉原工程的一些未用到的.c 文件，最终的工程如图所示：



usart3.c 在之前的例程（详见：PZ-HC05 蓝牙串口模块使用说明）已经有介绍过，这里，我们主要看 sim800c.c 和 main.c 的代码。

3.1 sim800c.c

首先是 sim800c.c，该文件是 PZ-SIM800C 模块的驱动代码，sim800c.c 里面的代码如下：

```
1.  #include "sim800c.h"
2.  #include "usart.h"
3.  #include "SysTick.h"
4.  #include "led.h"
5.  #include "key.h"
```

```

6.  #include "tftlcd.h"
7.  #include "flash.h"
8.  #include "touch.h"
9.  #include "malloc.h"
10. #include "string.h"
11. #include "font_show.h"
12. #include "usart3.h"
13. #include "ff.h"
14. #include "time.h"
15.
16.
17.
18.  u8 Scan_Wtime = 0; //保存蓝牙扫描需要的时间
19.  u8 BT_Scan_mode=0; //蓝牙扫描设备模式标志
20.
21.  //usmart 支持部分
22.  //将收到的 AT 指令应答数据返回给电脑串口
23.  //mode:0,不清零 USART3_RX_STA;
24.  //    1,清零 USART3_RX_STA;
25.  void sim_at_response(u8 mode)
26.  {
27.      if(USART3_RX_STA&0X8000) //接收到一次数据了
28.      {
29.          USART3_RX_BUF[USART3_RX_STA&0X7FFF]=0; //添加结束符
30.          printf("%s",USART3_RX_BUF); //发送到串口
31.          if(mode)USART3_RX_STA=0;
32.      }
33.  }
34.
35.  //PZ-SIM800C 各项测试(拨号测试、短信测试、GPRS 测试,蓝牙测试)共用代码
36.
37.  //SIM800C 发送命令后,检测接收到的应答
38.  //str:期待的应答结果
39.  //返回值:0,没有得到期待的应答结果
40.  //其他,期待应答结果的位置(str 的位置)
41.  u8* sim800c_check_cmd(u8 *str)
42.  {
43.      char *strx=0;
44.      if(USART3_RX_STA&0X8000) //接收到一次数据了
45.      {
46.          USART3_RX_BUF[USART3_RX_STA&0X7FFF]=0; //添加结束符
47.          strx=strstr((const char*)USART3_RX_BUF,(const char*)str);
48.      }
49.      return (u8*)strx;

```

```

50. }
51. //SIM800C 发送命令
52. //cmd:发送的命令字符串(不需要添加回车了),当 cmd<0XFF 的时候,发送数字(比如发送 0X1A),大于的时候发送字
    符串.
53. //ack:期待的应答结果,如果为空,则表示不需要等待应答
54. //waittime:等待时间(单位:10ms)
55. //返回值:0,发送成功(得到了期待的应答结果)
56. //      1,发送失败
57. u8 sim800c_send_cmd(u8 *cmd,u8 *ack,u16 waittime)
58. {
59.     u8 res=0;
60.     USART3_RX_STA=0;
61.     if((u32)cmd<=0XFF)
62.     {
63.         while((USART3->SR&0X40)==0);//等待上一次数据发送完成
64.         USART3->DR=(u32)cmd;
65.     }else usart3_printf("%s\r\n",cmd);//发送命令
66.
67.     if(waittime==1100)//11s 后读回串口数据(蓝牙测试用到)
68.     {
69.         Scan_Wtime = 11;//需要定时的时间
70.         TIM7_SetARR(9999);//产生 1S 定时中断
71.
72.     }
73.     if(ack&&waittime) //需要等待应答
74.     {
75.         while(--waittime) //等待倒计时
76.         {
77.             if(BT_Scan_mode)//蓝牙测试模式下
78.             {
79.                 res=KEY_Scan(0);//返回上一级
80.                 if(res==KEY1_PRESS)return 2;
81.             }
82.             delay_ms(10);
83.             if(USART3_RX_STA&0X8000)//接收到期待的应答结果
84.             {
85.                 if(sim800c_check_cmd(ack))break;//得到有效数据
86.                 USART3_RX_STA=0;
87.             }
88.         }
89.         if(waittime==0)res=1;
90.     }
91.     return res;
92. }

```

```

93. //接收 SIM800C 返回数据（蓝牙测试模式下使用）
94. //request:期待接收命令字符串
95. //waitting:等待时间(单位: 10ms)
96. //返回值:0,发送成功(得到了期待的应答结果)
97. //      1,发送失败
98. u8 sim800c_wait_request(u8 *request ,u16 waittime)
99. {
100.     u8 res = 1;
101.     u8 key;
102.     if(request && waittime)
103.     {
104.         while(--waittime)
105.         {
106.             key=KEY_Scan(0);
107.             if(key==KEY1_PRESS) return 2;//返回上一级
108.             delay_ms(10);
109.             if(USART3_RX_STA &0x8000)//接收到期待的应答结果
110.             {
111.                 if(sim800c_check_cmd(request)) break;//得到有效数据
112.                 USART3_RX_STA=0;
113.             }
114.         }
115.         if(waittime==0)res=0;
116.     }
117.     return res;
118. }
119.
120. //将 1 个字符转换为 16 进制数字
121. //chr:字符,0~9/A~F/a~f
122. //返回值:chr 对应的 16 进制数值
123. u8 sim800c_chr2hex(u8 chr)
124. {
125.     if(chr>='0'&&chr<='9')return chr-'0';
126.     if(chr>='A'&&chr<='F')return (chr-'A'+10);
127.     if(chr>='a'&&chr<='f')return (chr-'a'+10);
128.     return 0;
129. }
130. //将 1 个 16 进制数字转换为字符
131. //hex:16 进制数字,0~15;
132. //返回值:字符
133. u8 sim800c_hex2chr(u8 hex)
134. {
135.     if(hex<=9)return hex+'0';
136.     if(hex>=10&&hex<=15)return (hex-10+'A');

```

```

137.     return '0';
138. }
139. //unicode gbk 转换函数
140. //src:输入字符串
141. //dst:输出(uni2gbk 时为 gbk 内码,gbk2uni 时,为 unicode 字符串)
142. //mode:0,unicode 到 gbk 转换;
143. //     1,gbk 到 unicode 转换;
144. void sim800c_unigbk_exchange(u8 *src,u8 *dst,u8 mode)
145. {
146.     u16 temp;
147.     u8 buf[2];
148.     if(mode)//gbk 2 unicode
149.     {
150.         while(*src!=0)
151.         {
152.             if(*src<0X81)    //非汉字
153.             {
154.                 temp=(u16)ff_convert((WCHAR)*src,1);
155.                 src++;
156.             }else           //汉字,占 2 个字节
157.             {
158.                 buf[1]=*src++;
159.                 buf[0]=*src++;
160.                 temp=(u16)ff_convert((WCHAR)*(u16*)buf,1);
161.             }
162.             *dst++=sim800c_hex2chr((temp>>12)&0X0F);
163.             *dst++=sim800c_hex2chr((temp>>8)&0X0F);
164.             *dst++=sim800c_hex2chr((temp>>4)&0X0F);
165.             *dst++=sim800c_hex2chr(temp&0X0F);
166.         }
167.     }else //unicode 2 gbk
168.     {
169.         while(*src!=0)
170.         {
171.             buf[1]=sim800c_chr2hex(*src++)*16;
172.             buf[1]+=sim800c_chr2hex(*src++);
173.             buf[0]=sim800c_chr2hex(*src++)*16;
174.             buf[0]+=sim800c_chr2hex(*src++);
175.             temp=(u16)ff_convert((WCHAR)*(u16*)buf,0);
176.             if(temp<0X80){*dst=temp;dst++;}
177.             else {*(u16*)dst=swap16(temp);dst+=2;}
178.         }
179.     }
180.     *dst=0;//添加结束符

```

```

181. }
182. //键盘码表
183. const u8* kbd_tbl1[13]={ "1", "2", "3", "4", "5", "6", "7", "8", "9", "*", "0", "#", "DEL"};
184. const u8* kbd_tbl2[13]={ "1", "2", "3", "4", "5", "6", "7", "8", "9", ".", "0", "#", "DEL"};
185. u8** kbd_tbl;
186. u8* kbd_fn_tbl[2];
187. //加载键盘界面（尺寸为 240*140）
188. //x,y:界面起始坐标（320*240 分辨率的时候，x 必须为 0）
189. void sim800c_load_keyboard(u16 x,u16 y,u8 **kbtbl)
190. {
191.     u16 i;
192.     FRONT_COLOR=RED;
193.     kbd_tbl=kbtbl;
194.     LCD_Fill(x,y,x+240,y+140,WHITE);
195.     LCD_DrawRectangle(x,y,x+240,y+140);
196.     LCD_DrawRectangle(x+80,y,x+160,y+140);
197.     LCD_DrawRectangle(x,y+28,x+240,y+56);
198.     LCD_DrawRectangle(x,y+84,x+240,y+112);
199.     FRONT_COLOR=BLUE;
200.     for(i=0;i<15;i++)
201.     {
202.         if(i<13)LCD_ShowFontStringMid(x+(i%3)*80,y+6+28*(i/3),(u8*)kbd_tbl[i],16,80);
203.         else LCD_ShowFontStringMid(x+(i%3)*80,y+6+28*(i/3),kbd_fn_tbl[i-13],16,80);
204.     }
205. }
206. //按键状态设置
207. //x,y:键盘坐标
208. //key:键值（0~8）
209. //sta:状态，0，松开；1，按下；
210. void sim800c_key_staset(u16 x,u16 y,u8 keyx,u8 sta)
211. {
212.     u16 i=keyx/3,j=keyx%3;
213.     if(keyx>15)return;
214.     if(sta)LCD_Fill(x+j*80+1,y+i*28+1,x+j*80+78,y+i*28+26,GREEN);
215.     else LCD_Fill(x+j*80+1,y+i*28+1,x+j*80+78,y+i*28+26,WHITE);
216.     if(j&&(i>3))LCD_ShowFontStringMid(x+j*80,y+6+28*i,(u8*)kbd_fn_tbl[keyx-13],16,80);
217.     else LCD_ShowFontStringMid(x+j*80,y+6+28*i,(u8*)kbd_tbl[keyx],16,80);
218. }
219. //得到触摸屏的输入
220. //x,y:键盘坐标
221. //返回值：按键键值（1~15 有效；0,无效）
222. u8 sim800c_get_keynum(u16 x,u16 y)
223. {
224.     u16 i,j;

```

```

225.     static u8 key_x=0; //0,没有任何按键按下; 1~15, 1~15 号按键按下
226.     u8 key=0;
227.     tp_dev.scan(0);
228.     if(tp_dev.sta&TP_PRES_DOWN) //触摸屏被按下
229.     {
230.         for(i=0;i<5;i++)
231.         {
232.             for(j=0;j<3;j++)
233.             {
234.                 if(tp_dev.x[0]<(x+j*80+80)&&tp_dev.x[0]>(x+j*80)&&tp_dev.y[0]<(y+i*28+28)&&
                    tp_dev.y[0]>(y+i*28))
235.                 {
236.                     key=i*3+j+1;
237.                     break;
238.                 }
239.             }
240.             if(key)
241.             {
242.                 if(key_x==key)key=0;
243.                 else
244.                 {
245.                     sim800c_key_staset(x,y,key_x-1,0);
246.                     key_x=key;
247.                     sim800c_key_staset(x,y,key_x-1,1);
248.                 }
249.                 break;
250.             }
251.         }
252.     }else if(key_x)
253.     {
254.         sim800c_key_staset(x,y,key_x-1,0);
255.         key_x=0;
256.     }
257.     return key;
258. }
259. //////////////////////////////////////
    //////////////////////////////////////
260. //拨号测试部分代码
261.
262. //SIM800C 拨号测试
263. //用于拨打电话和接听电话
264. //返回值:0,正常
265. //其他,错误代码
266. u8 sim800c_call_test(void)

```

```

267. {
268.     u8 key;
269.     u16 lenx;
270.     u8 callbuf[20];
271.     u8 pohnenumlen=0;    //号码长度,最大 15 个数
272.     u8 *p,*p1,*p2;
273.     u8 oldmode=0;
274.     u8 cmode=0; //模式
275.         //0:等待拨号
276.         //1:拨号中
277.         //2:通话中
278.         //3:接收到来电
279.     LCD_Clear(WHITE);
280.     if(sim800c_send_cmd("AT+CTTSRING=0","OK",200))return 1; //设置 TTS 来电设置 0: 来电有铃声 1:
        没有
281.     if(sim800c_send_cmd("AT+CTTSPARAM=20,0,50,70,0","OK",200))return 1; //设置 TTS 声音大小、语
        调配置
282.     if(sim800c_send_cmd("AT+CLIP=1","OK",200))return 1; //设置来电显示
283.     if(sim800c_send_cmd("AT+COLP=1","OK",200))return 2; //设置被叫号码显示
284.     p1=mymalloc(SRAMIN,20); //申请 20 直接用于存放号码
285.     if(p1==NULL)return 2;
286.     FRONT_COLOR=RED;
287.     LCD_ShowFontStringMid(0,30,"PZ-SIM800C 拨号测试",16,240);
288.     LCD_ShowFontString(40,70,200,16,"请拨号:",16,0);
289.     kbd_fn_tbl[0]="拨号";
290.     kbd_fn_tbl[1]="返回";
291.     sim800c_load_keyboard(0,180,(u8**)kbd_tbl1);
292.     FRONT_COLOR=BLUE;
293.     while(1)
294.     {
295.         delay_ms(10);
296.         if(USART3_RX_STA&0X8000) //接收到数据
297.         {
298.             sim_at_response(0);
299.             if(cmode==1||cmode==2)
300.             {
301.                 if(cmode==1)if(sim800c_check_cmd("+COLP:"))cmode=2; //拨号成功
302.                 if(sim800c_check_cmd("NO CARRIER"))cmode=0; //拨号失败
303.                 if(sim800c_check_cmd("NO ANSWER"))cmode=0; //拨号失败
304.                 if(sim800c_check_cmd("ERROR"))cmode=0; //拨号失败
305.             }
306.             if(sim800c_check_cmd("+CLIP:"))//接收到来电
307.             {
308.                 cmode=3;

```

```

309.         p=sim800c_check_cmd("+CLIP:");
310.         p+=8;
311.         p2=(u8*)strstr((const char *)p, "\"");
312.         p2[0]=0; //添加结束符
313.         strcpy((char*)p1, (char*)p);
314.     }
315.     USART3_RX_STA=0;
316. }
317. key=sim800c_get_keynum(0,180);
318. if(key)
319. {
320.     if(key<13)
321.     {
322.         if(cmode==0&&pohnenumlen<15)
323.         {
324.             callbuf[pohnenumlen++]=kbd_tbl[key-1][0];
325.             usart3_printf("AT+CLDTMF=2,\"%c\"\\r\\n",kbd_tbl[key-1][0]);
326.             delay_ms(55);
327.             usart3_printf("AT+CTTS=2,\"%c\"\\r\\n",kbd_tbl[key-1][0]); //TTS 语
音
328.         }else if(cmode==2)//通话中
329.         {
330.             usart3_printf("AT+CLDTMF=2,\"%c\"\\r\\n",kbd_tbl[key-1][0]);
331.             delay_ms(100);
332.             usart3_printf("AT+VTS=%c\\r\\n",kbd_tbl[key-1][0]);
333.             LCD_ShowChar(40+56,90,kbd_tbl[key-1][0],16,0);
334.         }
335.     }else
336.     {
337.         if(key==13)if(pohnenumlen&&cmode==0)pohnenumlen--; //删除
338.         if(key==14)//执行拨号
339.         {
340.             if(cmode==0)//拨号模式
341.             {
342.                 callbuf[pohnenumlen]=0; //最后加入结束符
343.                 usart3_printf("ATD%s;\\r\\n",callbuf);//拨号
344.                 cmode=1; //拨号中模式
345.             }else
346.             {
347.                 sim800c_send_cmd("ATH","OK",200);//挂机
348.                 cmode=0;
349.             }
350.         }
351.         if(key==15)

```

```

352.         {
353.             if(cmode==3)//接收到来电
354.             {
355.                 sim800c_send_cmd("ATA", "OK", 200); //发送应答指令
356.                 LCD_ShowFontString(40+56, 70, 200, 16, callbuf, 16, 0);
357.                 cmode=2;
358.             }else
359.             {
360.                 sim800c_send_cmd("ATH", 0, 0); //不管有没有在通话,都结束通话
361.                 break; //退出循环
362.             }
363.         }
364.     }
365.     if(cmode==0)//只有在等待拨号模式有效
366.     {
367.         callbuf[pohnumlen]=0;
368.         LCD_Fill(40+56, 70, 239, 70+16, WHITE);
369.         LCD_ShowFontString(40+56, 70, 200, 16, callbuf, 16, 0);
370.     }
371. }
372. if(oldmode!=cmode)//模式变化了
373. {
374.     switch(cmode)
375.     {
376.         case 0:
377.             kbd_fn_tbl[0]="拨号";
378.             kbd_fn_tbl[1]="返回";
379.             FRONT_COLOR=RED;
380.             LCD_ShowFontString(40, 70, 200, 16, "请拨号:", 16, 0);
381.             LCD_Fill(40+56, 70, 239, 70+16, WHITE);
382.             if(pohnumlen)
383.             {
384.                 FRONT_COLOR=BLUE;
385.                 LCD_ShowFontString(40+56, 70, 200, 16, callbuf, 16, 0);
386.             }
387.             break;
388.         case 1:
389.             FRONT_COLOR=RED;
390.             LCD_ShowFontString(40, 70, 200, 16, "拨号中:", 16, 0);
391.             pohnumlen=0;
392.         case 2:
393.             FRONT_COLOR=RED;
394.             if(cmode==2)LCD_ShowFontString(40, 70, 200, 16, "通话中:", 16, 0);
395.             kbd_fn_tbl[0]="挂断";

```

```

396.         kbd_fn_tbl[1]="返回";
397.         break;
398.     case 3:
399.         FRONT_COLOR=RED;
400.         LCD_ShowFontString(40,70,200,16,"有来电:",16,0);
401.         FRONT_COLOR=BLUE;
402.         LCD_ShowFontString(40+56,70,200,16,p1,16,0);
403.         kbd_fn_tbl[0]="挂断";
404.         kbd_fn_tbl[1]="接听";
405.         break;
406.     }
407.     if(cmode==2)LCD_ShowFontString(40,90,200,16,"DTMF 音:",16,0); //通话中,可以通过键
        盘输入 DTMF 音
408.     else LCD_Fill(40,90,120,90+16,WHITE);
409.     sim800c_load_keyboard(0,180,(u8**)kbd_tbl1); //显示键盘
410.     oldmode=cmode;
411. }
412. if((lenx%50)==0)LED1=!LED1;
413. lenx++;
414. }
415. myfree(SRAMIN,p1);
416. return 0;
417. }
418. ///////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
    ///////////////////////////////////////////////////////////////////
419. //短信测试部分代码
420.
421. //SIM800C 读短信测试
422. void sim800c_sms_read_test(void)
423. {
424.     u8 *p,*p1,*p2;
425.     u8 timex=0;
426.     u8 msgindex[3];
427.     u8 msglen=0;
428.     u8 msgmaxnum=0; //短信最大条数
429.     u8 key=0;
430.     u8 smsreadsta=0; //是否在短信显示状态
431.     p=mymalloc(SRAMIN,200);//申请 200 个字节的内存
432.     LCD_Clear(WHITE);
433.     FRONT_COLOR=RED;
434.     LCD_ShowFontStringMid(0,30,"PZ-SIM800C 读短信测试",16,240);
435.     LCD_ShowFontString(30,50,200,16,"读取:    总信息:",16,0);
436.     kbd_fn_tbl[0]="读取";
437.     kbd_fn_tbl[1]="返回";

```

```

438.     sim800c_load_keyboard(0,180,(u8**)kbd_tbl1); //显示键盘
439.     while(1)
440.     {
441.         key=sim800c_get_keynum(0,180);
442.         if(key)
443.         {
444.             if(smsreadsta)
445.             {
446.                 LCD_Fill(30,75,239,179,WHITE); //清除显示的短信内容
447.                 usart3_printf("AT+CTTS=0\r\n");
448.                 smsreadsta=0;
449.             }
450.             if(key<10||key==11)
451.             {
452.                 if(msglen<2)
453.                 {
454.                     msgindex[msglen++]=kbd_tbl[key-1][0];
455.                     usart3_printf("AT+CLDTMF=2,\"%c\"\r\n",kbd_tbl[key-1][0]);
456.                 }
457.                 if(msglen==2)
458.                 {
459.                     key=(msgindex[0]-'0')*10+msgindex[1]-'0';
460.                     if(key>msgmaxnum)
461.                     {
462.                         msgindex[0]=msgmaxnum/10+'0';
463.                         msgindex[1]=msgmaxnum%10+'0';
464.                     }
465.                 }
466.             }else
467.             {
468.                 if(key==10||key==12) usart3_printf("AT+CTTS=0\r\n");
469.                 if(key==13)
470.                 {
471.                     usart3_printf("AT+CTTS=0\r\n");
472.                     if(msglen)msglen--; //删除
473.                 }
474.                 if(key==14&&msglen) //执行读取短信
475.                 {
476.                     LCD_Fill(30,75,239,179,WHITE); //清除之前的显示
477.                     sprintf((char*)p,"AT+CMGR=%s",msgindex);
478.                     if(sim800c_send_cmd(p,"+CMGR:",200)==0) //读取短信
479.                     {
480.                         FRONT_COLOR=RED;
481.                         LCD_ShowFontString(30,75,200,12,"状态:",12,0);

```

```

482. LCD_ShowFontString(30+75,75,200,12,"来自:",12,0);
483. LCD_ShowFontString(30,90,200,12,"接收时间:",12,0);
484. LCD_ShowFontString(30,105,200,12,"内容:",12,0);
485. FRONT_COLOR=BLUE;
486. if(strstr((const char*)(USART3_RX_BUF),"UNREAD")==0)LCD_ShowFontStr
    ing(30+30,75,200,12,"已读",12,0);
487. else LCD_ShowFontString(30+30,75,200,12,"未读",12,0);
488. p1=(u8*)strstr((const char*)(USART3_RX_BUF),",");
489. p2=(u8*)strstr((const char*)(p1+2),"\n");
490. p2[0]=0;//加入结束符
491. sim800c_unigbk_exchange(p1+2,p,0); //将 unicode 字符转换为 gbk
    码
492. LCD_ShowFontString(30+75+30,75,200,12,p,12,0); //显示电话号码
493. p1=(u8*)strstr((const char*)(p2+1),"/");
494. p2=(u8*)strstr((const char*)(p1),"+");
495. p2[0]=0;//加入结束符
496. LCD_ShowFontString(30+54,90,200,12,p1-2,12,0); //显示接收时
    间
497. p1=(u8*)strstr((const char*)(p2+1),"\r"); //寻找回车符
498. sim800c_unigbk_exchange(p1+2,p,0); //将 unicode 字符转换为 gbk
    码
499. LCD_ShowFontString(30+30,105,180,75,p,12,0); //显示短信内
    容
500. usart3_printf("AT+CTTS=2,\"%s\"\\r\\n",p);//TTS 读取短信 ASCII 模式
501. smsreadsta=1; //标记有显示短信内容
502. }else
503. {
504. LCD_ShowFontString(30,75,200,12,"无短信内容!!!请检查!!",12,0);
505. usart3_printf("AT+CTTS=2,\"无短信内容请检查\"\\r\\n");
506. delay_ms(1000);
507. delay_ms(1000);
508. LCD_Fill(30,75,239,75+12,WHITE);//清除显示
509. }
510. USART3_RX_STA=0;
511. }
512. if(key==15){usart3_printf("AT+CTTS=0\\r\\n");break;}
513. }
514. msgindex[msglen]=0;
515. LCD_Fill(30+40,50,86,50+16,WHITE);
516. LCD_ShowFontString(30+40,50,86,16,msgindex,16,0);
517. }
518. if(timex==0) //2.5 秒左右更新一次
519. {
520. if(sim800c_send_cmd("AT+CPMS?", "+CPMS:",200)==0) //查询优选消息存储器

```

```

521.         {
522.             p1=(u8*)strstr((const char*)(USART3_RX_BUF),",");
523.             p2=(u8*)strstr((const char*)(p1+1),",");
524.             p2[0]='/';
525.             if(p2[3]=='')//小于 64K SIM 卡，最多存储几十条短信
526.             {
527.                 msgmaxnum=(p2[1]-'0')*10+p2[2]-'0';    //获取最大存储短信条数
528.                 p2[3]=0;
529.             }else //如果是 64K SIM 卡，则能存储 100 条以上的信息
530.             {
531.                 msgmaxnum=(p2[1]-'0')*100+(p2[2]-'0')*10+p2[3]-'0';//获取最大存储短信条
数
532.                 p2[4]=0;
533.             }
534.             sprintf((char*)p,"%s",p1+1);
535.             LCD_ShowFontString(30+17*8,50,200,16,p,16,0);
536.             USART3_RX_STA=0;
537.         }
538.     }
539.     if((timex%20)==0)LED1=!LED1;//200ms 闪烁
540.     timex++;
541.     delay_ms(10);
542.     if(USART3_RX_STA&0X8000)sim_at_response(1);//检查从 GSM 模块接收到的数据
543. }
544. myfree(SRAMIN,p);
545. }
546. //测试短信发送内容(70 个字[UCS2 的时候,1 个字符/数字都算 1 个字])
547. const u8* sim800c_test_msg="您好，这是一条测试短信，由 PZ-SIM800C GSM/GPRS 模块发送";
548. //SIM800C 发短信测试
549. void sim800c_sms_send_test(void)
550. {
551.     u8 *p,*p1,*p2;
552.     u8 phonebuf[20];    //号码缓存
553.     u8 pohnenumlen=0;    //号码长度,最大 15 个数
554.     u8 timex=0;
555.     u8 key=0;
556.     u8 smssendsta=0;    //短信发送状态,0,等待发送;1,发送失败;2,发送成功
557.     p=mymalloc(SRAMIN,100); //申请 100 个字节的内存,用于存放电话号码的 unicode 字符串
558.     p1=mymalloc(SRAMIN,300); //申请 300 个字节的内存,用于存放短信的 unicode 字符串
559.     p2=mymalloc(SRAMIN,100); //申请 100 个字节的内存 存放: AT+CMGS=p1
560.     LCD_Clear(WHITE);
561.     FRONT_COLOR=RED;
562.     LCD_ShowFontStringMid(0,30,"PZ-SIM800C 发短信测试",16,240);
563.     LCD_ShowFontString(30,50,200,16,"发送给:",16,0);

```

```

564.    LCD_ShowFontString(30,70,200,16,"状态:",16,0);
565.    LCD_ShowFontString(30,90,200,16,"内容:",16,0);
566.    FRONT_COLOR=BLUE;
567.    LCD_ShowFontString(30+40,70,170,90,"等待发送",16,0);//显示状态
568.    LCD_ShowFontString(30+40,90,170,90,(u8*)sim800c_test_msg,16,0);//显示短信内容
569.    kbd_fn_tbl[0]="发送";
570.    kbd_fn_tbl[1]="返回";
571.    sim800c_load_keyboard(0,180,(u8**)kbd_tbl1);//显示键盘
572.    while(1)
573.    {
574.        key=sim800c_get_keynum(0,180);
575.        if(key)
576.        {
577.            if(smssendsta)
578.            {
579.                smssendsta=0;
580.                LCD_ShowFontString(30+40,70,170,90,"等待发送",16,0);//显示状态
581.            }
582.            if(key<10||key==11)
583.            {
584.                if(pohnenumlen<15)
585.                {
586.                    phonebuf[pohnenumlen++]=kbd_tbl[key-1][0];
587.                    usart3_printf("AT+CLDTMF=2,\"%c\"\\r\\n",kbd_tbl[key-1][0]);
588.                }
589.            }else
590.            {
591.                if(key==13)if(pohnenumlen)pohnenumlen--;//删除
592.                if(key==14&&pohnenumlen)                //执行发送短信
593.                {
594.                    LCD_ShowFontString(30+40,70,170,90,"正在发送",16,0);                //显示正在发送
595.                    smssendsta=1;
596.                    sim800c_unigbk_exchange(phonebuf,p,1);                //将电话号码转换为
                    unicode 字符串
597.                    sim800c_unigbk_exchange((u8*)sim800c_test_msg,p1,1);//将短信内容转换为
                    unicode 字符串.
598.                    sprintf((char*)p2,"AT+CMGS=\"%s\"",p);
599.                    if(sim800c_send_cmd(p2,">",200)==0)                //发送短信命令+电话
                        号码
600.                    {
601.                        usart3_printf("%s",p1);                //发送短信内容到
                        GSM 模块

```

```

602.             if(sim800c_send_cmd((u8*)0X1A, "+CMGS:",1000)==0)smssendsta=2;//发送
                结束符,等待发送完成(最长等待 10 秒钟,因为短信长了的话,等待时间会长一些)
603.             }
604.             if(smssendsta==1)LCD_ShowFontString(30+40,70,170,90,"发送失败
                ",16,0);    //显示状态
605.             else LCD_ShowFontString(30+40,70,170,90,"发送成功
                ",16,0);    //显示状态
606.             USART3_RX_STA=0;
607.             }
608.             if(key==15)break;
609.             }
610.             phonebuf[pohnumlen]=0;
611.             LCD_Fill(30+54,50,239,50+16,WHITE);
612.             LCD_ShowFontString(30+54,50,156,16,phonebuf,16,0);
613.             }
614.             if((timex%20)==0)LED1=!LED1;//200ms 闪烁
615.             timex++;
616.             delay_ms(10);
617.             if(USART3_RX_STA&0X8000)sim_at_response(1);//检查从 GSM 模块接收到的数据
618.             }
619.             myfree(SRAMIN,p);
620.             myfree(SRAMIN,p1);
621.             myfree(SRAMIN,p2);
622.             }
623.             //sms 测试主界面
624.             void sim800c_sms_ui(u16 x,u16 y)
625.             {
626.                 LCD_Clear(WHITE);
627.                 FRONT_COLOR=RED;
628.                 LCD_ShowFontStringMid(0,y,"PZ-SIM800C 短信测试",16,240);
629.                 LCD_ShowFontString(x,y+40,200,16,"请选择:",16,0);
630.                 LCD_ShowFontString(x,y+60,200,16,"KEY0:读短信测试",16,0);
631.                 LCD_ShowFontString(x,y+80,200,16,"KEY1:发短信测试",16,0);
632.                 LCD_ShowFontString(x,y+100,200,16,"KEY_UP:返回上级菜单",16,0);
633.             }
634.             //SIM800C 短信测试
635.             //用于读短信或者发短信
636.             //返回值:0,正常
637.             //其他,错误代码
638.             u8 sim800c_sms_test(void)
639.             {
640.                 u8 key;
641.                 u8 timex=0;
642.                 if(sim800c_send_cmd("AT+CMGF=1","OK",200))return 1;    //设置文本模式

```

```

643.     if(sim800c_send_cmd("AT+CSCS=\"UCS2\"", "OK", 200))return 2;    //设置 TE 字符集为 UCS2
644.     if(sim800c_send_cmd("AT+CSMP=17,0,2,25", "OK", 200))return 3;    //设置短消息文本模式参数
645.     if(sim800c_send_cmd("AT+CTTSPARAM=5,0,51,61,0", "OK", 200))return 1;    //设置 TTS 声音大小、语
        调配置
646.     sim800c_sms_ui(40,30);
647.     while(1)
648.     {
649.         key=KEY_Scan(0);
650.         if(key==KEY0_PRESS)
651.         {
652.             sim800c_sms_read_test();
653.             sim800c_sms_ui(40,30);
654.             timex=0;
655.         }else if(key==KEY1_PRESS)
656.         {
657.             sim800c_sms_send_test();
658.             sim800c_sms_ui(40,30);
659.             timex=0;
660.         }else if(key==KEY_UP_PRESS)break;
661.         timex++;
662.         if(timex==20)
663.         {
664.             timex=0;
665.             LED1=!LED1;
666.         }
667.         delay_ms(10);
668.         sim_at_response(1);    //检查 GSM 模块发送过来的数据,
        及时上传给电脑
669.     }
670.     sim800c_send_cmd("AT+CSCS=\"GSM\"", "OK", 200);    //设置默认的 GSM 7 位缺省字符
        集
671.     return 0;
672. }
673. //////////////////////////////////////
        //////////////////////////////////////
674. //GPRS 测试部分代码
675.
676. const u8 *modetb1[2]={"TCP", "UDP"}; //连接模式
677. //tcp/udp 测试
678. //带心跳功能,以维持连接
679. //mode:0:TCP 测试;1,UDP 测试)
680. //ipaddr:ip 地址
681. //port:端口
682. void sim800c_tcpudp_test(u8 mode,u8* ipaddr,u8* port)

```

```

683. {
684.     u8 *p,*p1,*p2,*p3;
685.     u8 key;
686.     u16 timex=0;
687.     u8 count=0;
688.     const u8* cnttbl[3]={"正在连接","连接成功","连接关闭"};
689.     u8 connectsta=0;           //0,正在连接;1,连接成功;2,连接关闭;
690.     u8 hbeaterrcnt=0;         //心跳错误计数器,连续5次心跳信号无应答,则重新连接
691.     u8 oldsta=0XFF;
692.     p=mymalloc(SRAMIN,100);   //申请100字节内存
693.     p1=mymalloc(SRAMIN,100);  //申请100字节内存
694.     LCD_Clear(WHITE);
695.     FRONT_COLOR=RED;
696.     if(mode)LCD_ShowFontStringMid(0,30,"PZ-SIM800C UDP 连接测试",16,240);
697.     else LCD_ShowFontStringMid(0,30,"PZ-SIM800C TCP 连接测试",16,240);
698.     LCD_ShowFontString(30,50,200,16,"KEY_UP:退出测试 KEY0:发送数据",12,0);
699.     sprintf((char*)p,"IP 地址:%s 端口:%s",ipaddr,port);
700.     LCD_ShowFontString(30,65,200,12,p,12,0);           //显示 IP 地址和端口
701.     LCD_ShowFontString(30,80,200,12,"状态:",12,0);     //连接状态
702.     LCD_ShowFontString(30,100,200,12,"发送数据:",12,0); //连接状态
703.     LCD_ShowFontString(30,115,200,12,"接收数据:",12,0); //端口固定为8086
704.     FRONT_COLOR=BLUE;
705.     USART3_RX_STA=0;
706.     sprintf((char*)p,"AT+CIPSTART=\"%s\", \"%s\", \"%s\"",modetbl[mode],ipaddr,port);
707.     if(sim800c_send_cmd(p,"OK",500))return;           //发起连接
708.     while(1)
709.     {
710.         key=KEY_Scan(0);
711.         if(key==KEY_UP_PRESS)//退出测试
712.         {
713.             sim800c_send_cmd("AT+CIPCLOSE=1","CLOSE OK",500); //关闭连接
714.             sim800c_send_cmd("AT+CIPSHUT","SHUT OK",500);     //关闭移动场景
715.             break;
716.         }else if(key==KEY0_PRESS&(hbeaterrcnt==0))           //发送数据(心跳正常时发送)
717.         {
718.             LCD_ShowFontString(30+30,80,200,12,"数据发送中...",12,0); //提示数据发送
中
719.             if(sim800c_send_cmd("AT+CIPSEND",">",500)==0)    //发送数据
720.             {
721.                 printf("CIPSEND DATA:%s\r\n",p1);           //发送数据打印到串口
722.                 usart3_printf("%s\r\n",p1);
723.                 delay_ms(10);
724.                 if(sim800c_send_cmd((u8*)0X1A,"SEND OK",1000)==0)LCD_ShowFontString(30+30,8
0,200,12,"数据发送成功!",12,0); //最长等待10s

```

```

725.         else LCD_ShowFontString(30+30,80,200,12,"数据发送失败!",12,0);
726.         delay_ms(1000);
727.     }else sim800c_send_cmd((u8*)0X1B,0,0);  //ESC,取消发送
728.     oldsta=0XFF;
729. }
730. if((timex%20)==0)
731. {
732.     LED1=!LED1;
733.     count++;
734.     if(connectsta==2||hbeaterrcnt>8)//连接中断了,或者连续 8 次心跳没有正确发送成功,则重新
        连接
735.     {
736.         sim800c_send_cmd("AT+CIPCLOSE=1","CLOSE OK",500);  //关闭连接
737.         sim800c_send_cmd("AT+CIPSHUT","SHUT OK",500);      //关闭移动场景
738.         sim800c_send_cmd(p,"OK",500);                      //尝试重新连接
739.         connectsta=0;
740.         hbeaterrcnt=0;
741.     }
742.     sprintf((char*)p1,"PZ-SIM800C %s 测试 %d ",modetbl[mode],count);
743.     LCD_ShowFontString(30+54,100,200,12,p1,12,0);
744. }
745. if(connectsta==0&&(timex%200)==0)//连接还没建立的时候,每 2 秒查询一次 CIPSTATUS.
746. {
747.     sim800c_send_cmd("AT+CIPSTATUS","OK",500);  //查询连接状态
748.     if(strstr((const char*)USART3_RX_BUF,"CLOSED"))connectsta=2;
749.     if(strstr((const char*)USART3_RX_BUF,"CONNECT OK"))connectsta=1;
750. }
751. if(connectsta==1&&timex>=600)//连接正常的时候,每 6 秒发送一次心跳
752. {
753.     timex=0;
754.     if(sim800c_send_cmd("AT+CIPSEND",">",200)==0)//发送数据
755.     {
756.         sim800c_send_cmd((u8*)0X00,0,0);  //发送数据:0X00
757.         delay_ms(20);                      //必须加延时
758.         sim800c_send_cmd((u8*)0X1A,0,0);  //CTRL+Z,结束数据发送,启动一次传输
759.     }else sim800c_send_cmd((u8*)0X1B,0,0);  //ESC,取消发送
760.
761.     hbeaterrcnt++;
762.     printf("hbeaterrcnt:%d\r\n",hbeaterrcnt);//方便调试代码
763. }
764. delay_ms(10);
765. if(USART3_RX_STA&0X8000)  //接收到一次数据了
766. {
767.     USART3_RX_BUF[USART3_RX_STA&0X7FFF]=0;  //添加结束符

```

```

768.         printf("%s",USART3_RX_BUF);           //发送到串口
769.         if(hbeaterrcnt)                         //需要检测心跳应答
770.         {
771.             if(strstr((const char*)USART3_RX_BUF,"SEND OK"))hbeaterrcnt=0;//心跳正常
772.         }
773.         p2=(u8*)strstr((const char*)USART3_RX_BUF,"+IPD");
774.         if(p2)//接收到 TCP/UDP 数据
775.         {
776.             p3=(u8*)strstr((const char*)p2,",");
777.             p2=(u8*)strstr((const char*)p2,":");
778.             p2[0]=0;//加入结束符
779.             sprintf((char*)p1,"收到%s 字节,内容如下",p3+1);//接收到的字节数
780.             LCD_Fill(30+54,115,239,130,WHITE);
781.             FRONT_COLOR=BRED;
782.             LCD_ShowFontString(30+54,115,156,12,p1,12,0); //显示接收到的数据长度
783.             FRONT_COLOR=BLUE;
784.             LCD_Fill(30,130,210,319,WHITE);
785.             LCD_ShowFontString(30,130,180,190,p2+1,12,0); //显示接收到的数据
786.         }
787.         USART3_RX_STA=0;
788.     }
789.     if(oldsta!=connectsta)
790.     {
791.         oldsta=connectsta;
792.         LCD_Fill(30+30,80,239,80+12,WHITE);
793.         LCD_ShowFontString(30+30,80,200,12,(u8*)cnttbl[connectsta],12,0); //更新状态
794.     }
795.     timex++;
796. }
797. myfree(SRAMIN,p);
798. myfree(SRAMIN,p1);
799. }
800. //gprs 测试主界面
801. void sim800c_gprs_ui(void)
802. {
803.     LCD_Clear(WHITE);
804.     FRONT_COLOR=RED;
805.     LCD_ShowFontStringMid(0,30,"PZ-SIM800C GPRS 通信测试",16,240);
806.     LCD_ShowFontString(30,50,200,16,"KEY_UP:连接方式切换",16,0);
807.     LCD_ShowFontString(30,90,200,16,"连接方式:",16,0); //连接方式通过 KEY_UP 设置(TCP/UDP)
808.     LCD_ShowFontString(30,110,200,16,"IP 地址:",16,0); //IP 地址可以键盘设置
809.     LCD_ShowFontString(30,130,200,16,"端口:",16,0); //端口固定为 8086
810.     kbd_fn_tbl[0]="连接";
811.     kbd_fn_tbl[1]="返回";

```

```

812.     sim800c_load_keyboard(0,180,(u8**)kbd_tb12); //显示键盘
813. }
814. //SIM800C GPRS 测试
815. //用于测试 TCP/UDP 连接
816. //返回值:0,正常
817. //其他,错误代码
818. u8 sim800c_gprs_test(void)
819. {
820.     const u8 *port="8086"; //端口固定为 8086,当你的电脑 8086 端口被其他程序占用时,请修改为其他
        空闲端口
821.     u8 mode=0; //0,TCP 连接;1,UDP 连接
822.     u8 key;
823.     u8 timex=0;
824.     u8 ipbuf[16]; //IP 缓存
825.     u8 iplen=0; //IP 长度
826.     sim800c_gprs_ui(); //加载主界面
827.     LCD_ShowFontString(30+72,90,200,16,(u8*)modetbl[mode],16,0); //显示连接方式
828.     LCD_ShowFontString(30+40,130,200,16,(u8*)port,16,0); //显示端口
829.     sim800c_send_cmd("AT+CIPCLOSE=1","CLOSE OK",100); //关闭连接
830.     sim800c_send_cmd("AT+CIPSHUT","SHUT OK",100); //关闭移动场景
831.     if(sim800c_send_cmd("AT+CGCLASS=\"B\"", "OK",1000))return 1; //设置 GPRS 移动台
        类别为 B,支持包交换和数据交换
832.     if(sim800c_send_cmd("AT+CGDCONT=1,\"IP\", \"CMNET\"", "OK",1000))return 2; //设置 PDP 上下文,
        互联网接协议,接入点等信息
833.     if(sim800c_send_cmd("AT+CGATT=1", "OK",500))return 3; //附着 GPRS 业务
834.     if(sim800c_send_cmd("AT+CIPCSGP=1,\"CMNET\"", "OK",500))return 4; //设置为 GPRS 连接
        模式
835.     if(sim800c_send_cmd("AT+CIPHEAD=1", "OK",500))return 5; //设置接收数据显
        示 IP 头(方便判断数据来源)
836.     ipbuf[0]=0;
837.     while(1)
838.     {
839.         key=KEY_Scan(0);
840.         if(key==KEY_UP_PRESS)
841.         {
842.             mode=!mode; //连接模式切换
843.             LCD_ShowFontString(30+72,90,200,16,(u8*)modetbl[mode],16,0); //显示连接模式
844.         }
845.         key=sim800c_get_keynum(0,180);
846.         if(key)
847.         {
848.             if(key<12)
849.             {
850.                 if(iplen<15)

```

```

851.         {
852.             ipbuf[iplen++]=kbd_tbl[key-1][0];
853.             usart3_printf("AT+CLDTMF=2,\"%c\"\\r\\n",kbd_tbl[key-1][0]);
854.         }
855.     }else
856.     {
857.         if(key==13)if(iplen)iplen--;    //删除
858.         if(key==14&&iplen)                //执行 GPRS 连接
859.         {
860.             sim800c_tcpudp_test(mode,ipbuf,(u8*)port);
861.             sim800c_gprs_ui();            //加载主界面
862.             LCD_ShowFontString(30+72,90,200,16,(u8*)modetbl[mode],16,0);    //显示连
接模式
863.             LCD_ShowFontString(30+40,130,200,16,(u8*)port,16,0);//显示端口
864.             USART3_RX_STA=0;
865.         }
866.         if(key==15)break;
867.     }
868.     ipbuf[iplen]=0;
869.     LCD_Fill(30+56,110,239,110+16,WHITE);
870.     LCD_ShowFontString(30+56,110,200,16,ipbuf,16,0);    //显示 IP 地址
871. }
872. timex++;
873. if(timex==20)
874. {
875.     timex=0;
876.     LED1=!LED1;
877. }
878. delay_ms(10);
879. sim_at_response(1);//检查 GSM 模块发送过来的数据,及时上传给电脑
880. }
881. return 0;
882. }
883. ///////////////////////////////////////////////////
///////////////////////////////////////////////////
884. //蓝牙测试部分代码
885.
886. //蓝牙 SPP 测试主界面
887. void sim800c_spp_ui(u16 x,u16 y)
888. {
889.     LCD_Clear(WHITE);
890.     FRONT_COLOR=RED;
891.     LCD_ShowFontStringMid(0,y,"PZ-SIM800C 蓝牙测试",16,240);
892.     LCD_ShowFontString(x,y+40,200,16,"请选择:",16,0);

```

```

893.     LCD_ShowFontString(x,y+60,200,16,"KEY2:发起配对请求模式",16,0);
894.     LCD_ShowFontString(x,y+80,200,16,"KEY0:接收配对请求模式",16,0);
895.     LCD_ShowFontString(x,y+100,200,16,"KEY1:返回上一级",16,0);
896.
897. }
898.
899. //SIM800C 蓝牙连接模式测试
900. //用于模式连接和串口透传
901. //mode(模式):0,接收配对模式
902. //      1,寻找设备模式
903. //返回值:0,正常
904. //      其他,错误代码
905. u8 sim800c_spp_mode(u8 mode)
906. {
907.     u8 *p1,*p2;
908.     u8 key;
909.     u8 timex=1;
910.     u8 sendcnt=0;
911.     u8 sendbuf[20];
912.     u8 res;
913.
914.     LCD_Clear(WHITE);
915.     FRONT_COLOR=RED;
916.     LCD_ShowFontStringMid(0,20,"PZ-SIM800C 蓝牙测试",16,240);
917.     LCD_ShowFontString(40,60,200,16,"KEY1:返回上一级",16,0);
918.
919.     if(mode==0)//接收配对模式
920.     {
921.         do
922.         {
923.             LCD_ShowFontString(40,30+60,200,16,"等待连接请求...",16,0);
924.             res = sim800c_wait_request("+BTPAIRING:",600); //等待手机端蓝牙
连接请求 6s
925.             if(res==1) //手机端连接请
求
926.             {
927.                 delay_ms(10);
928.                 sim800c_send_cmd("AT+BTPAIR=1,1","BTPAIR:",500); //响应连接
929.             }
930.             else if(res==2) return 0; //按键返回上一
级
931.             LCD_ShowFontString(40,30+60,200,16,"",16,0);
932.             delay_ms(50);
933.             LED1=!LED1;

```

```

934.          }while(strstr((const char*)USART3_RX_BUF,"+BTPAIR: 1")==NULL); //判断是否匹配成
功
935.          USART3_RX_STA=0;
936.      }
937.      else if(mode==1)//寻找设备模式
938.      {
939.          BT_Scan_mode=1;//设置蓝牙设备扫描标志
940.          do
941.          {
942.              LCD_ShowFontString(40,30+60,200,16,"搜索设备中...",16,0);
943.              res = sim800c_send_cmd("AT+BTSCAN=1,11","+BTSCAN: 1",1100);//搜索附近的蓝牙
设备,搜索 11s(重新配置定时器分频系数,定时为 1S 中断, 蓝牙扫描结束后重新配置为 10ms 定时中断)
944.              if(res==2){BT_Scan_mode=0;return 0;} //按键返回上一
级
945.              LCD_ShowFontString(40,30+60,200,16,"          ",16,0);
946.              delay_ms(100);
947.              LED1=!LED1;
948.          }while(strstr((const char*)USART3_RX_BUF,"+BTSCAN: 0")==NULL); //判断是否扫描到
设备
949.          USART3_RX_STA=0;
950.          do
951.          {
952.              LCD_ShowFontString(40,30+60,200,16,"发现设备", 16,0);
953.              res = sim800c_send_cmd("AT+BTPAIR=0,1","+BTPAIRING:",500); //连接搜索到的第
一个设备
954.              if(res==2){BT_Scan_mode=0;return 0;} //按键返回上一
级
955.              delay_ms(100);
956.              LCD_ShowFontString(40,30+60,200,16,"正在连接中.....",16,0);
957.              sim800c_send_cmd("AT+BTPAIR=1,1","+BTPAIR:",500); //响应连接 如果
手机长期不确认匹配, SIM800C 端蓝牙 30S 后才会上报配对失败
958.          }while(strstr((const char*)USART3_RX_BUF,"+BTPAIR: 1")==NULL); //判断是否匹配成
功
959.          USART3_RX_STA=0;
960.          BT_Scan_mode=0; //清除蓝牙设备扫
描标志
961.      }
962.      LCD_ShowFontString(40,30+60,200,16,"蓝牙连接成功          ",16,0);
963.      do
964.      {
965.          if(!LED1)LCD_ShowFontString(40,120,200,16,"等待 SPP 连接",16,0);
966.          else LCD_ShowFontString(40,120,200,16,"          ",16,0);
967.          res = sim800c_wait_request("SPP",120); //等待手机端 SPP 连接请求
968.          if(res==2)return 0; //按键返回上一级

```

```

969.         else if(res==1) break; //SPP 连接成功
970.         LED1=!LED1;
971.     }while(1);
972.     if(!sim800c_send_cmd("AT+BTACPT=1","+BTCONNECT:",300))//应答手机端 spp 连接请求 3S
973.     {
974.         LCD_ShowFontString(40,120,200,16,"SPP 连接成功",16,0);
975.         delay_ms(1000);
976.         LCD_ShowFontString(40,120,200,16,"",16,0);
977.         LCD_ShowFontString(10,140,200,16,"发送数据:",16,0);
978.         LCD_ShowFontString(10,200,200,16,"接收数据:",16,0);
979.     }
980.     else
981.     {
982.         LCD_ShowFontString(40,120,200,16,"SPP 连接失败",16,0);
983.         return 0;//若一段时间内没有应答,则需要重新连接蓝牙!
984.     }
985.     while(1)
986.     {
987.         key = KEY_Scan(0);
988.         if(key == KEY1_PRESS) break;//按键返回上一级
989.         if(USART3_RX_STA&0x8000)
990.         {
991.             USART3_RX_BUF[USART3_RX_STA&0X7FFF]=0;//添加结束符
992.             USART3_RX_STA=0;
993.             p1 =(u8*)strstr((const char*)USART3_RX_BUF,"DATA: ");
994.             if(p1!=NULL)
995.             {
996.                 p2 = (u8*)strstr((const char *)p1,"\x0d\x0a");
997.                 if(p2!= NULL)
998.                 {
999.                     p2 =(u8*)strstr((const char *)p1,",");
1000.                     p1 =(u8*)strstr((const char *)p2+1,",");
1001.                     //printf("接收到的数据是: ");
1002.                     //printf("%s\r\n",p1+1);//打印到串口
1003.                     LCD_Fill(90,200,320,480,WHITE); //清除显
示
1004.                     LCD_ShowString(90,200,150,119,16,(u8*)(p1+1)); //显示接收
到的数据
1005.                 }
1006.             }
1007.         else
1008.         {
1009.             p1 =(u8*)strstr((const char*)USART3_RX_BUF,"+BTDISCONN: ");//判断是否
断开连接

```

```

1010.         if(p1!=NULL)
1011.         {
1012.                 LCD_ShowFontString(40,30+60,200,16," 蓝 牙 连 接 断
开          ",16,0);
1013.                 LCD_Fill(10,140,240,320,WHITE);//清屏
1014.                 delay_ms(1000);
1015.                 delay_ms(1000);
1016.                 delay_ms(1000);
1017.                 break;//退出
1018.         }
1019.     }
1020. }
1021.     timex++;
1022.     if(timex%50==0)
1023.     {
1024.         timex=0;
1025.         sim800c_send_cmd("AT+BTSPSEND", ">",100);           //发送数据
1026.         sprintf((char*)sendbuf,"Bluetooth test %d \r\n\32",sendcnt);
1027.         sendcnt++;
1028.         if(sendcnt>99) sendcnt = 0;
1029.         res = sim800c_send_cmd((u8*)sendbuf,"OK",100);       //发送数据
1030.         if(res==0)
1031.         {
1032.                 LCD_ShowString(90,140,209,119,16,(u8*)sendbuf); //显示发送的数
据
1033.                 LED1=!LED1;
1034.         }
1035.     else
1036.     {
1037.         LCD_ShowFontString(40,30+60,200,16,"蓝牙连接断开          ",16,0);
1038.         LCD_Fill(10,140,240,320,WHITE);//清屏
1039.         delay_ms(1000);
1040.         delay_ms(1000);
1041.         delay_ms(1000);
1042.         break;//退出
1043.     }
1044. }
1045.     delay_ms(10);
1046. }
1047.     return 0;
1048. }
1049.
1050. //SIM800C 蓝牙连接模式选择
1051. //返回值:0,正常

```

```

1052. //      其他,错误代码
1053. u8 sim800c_spp_test(void)
1054. {
1055.     u8 key;
1056.     u8 timex=0;
1057.     if(sim800c_send_cmd("ATE1","OK",200))//打开回显失败
1058.     {
1059.         // printf("打开回显失败");
1060.         return 1;
1061.     }
1062.     delay_ms(10);
1063.     if(sim800c_send_cmd("AT+BTPOWER=1","AT",300))//打开蓝牙电源 不判断 OK, 因为电源原本开启再发
        送打开的话会返回 error
1064.     {
1065.         sim800c_send_cmd("ATE0","OK",200);//关闭回显功能
1066.         return 1;
1067.     }
1068.     delay_ms(10);
1069.     sim800c_spp_ui(40,30);
1070.     while(1)
1071.     {
1072.         key=KEY_Scan(0);
1073.         if(key==KEY2_PRESS)//KEY2_PRESS
1074.         {
1075.             sim800c_spp_mode(1);//寻找设备模式
1076.             sim800c_spp_ui(40,30);
1077.             sim800c_send_cmd("AT+BTUNPAIR=0","AT",100);//删除配对信息
1078.             timex=0;
1079.         }
1080.         else if(key==KEY0_PRESS)
1081.         {
1082.             sim800c_spp_mode(0);//接收配对模式
1083.             sim800c_spp_ui(40,30);
1084.             sim800c_send_cmd("AT+BTUNPAIR=0","AT",100);//删除配对信息
1085.             timex=0;
1086.
1087.         }
1088.         else if(key==KEY1_PRESS)
1089.         {
1090.             sim800c_send_cmd("ATE0","OK",300);//关闭回显功能
1091.             break;
1092.         }
1093.
1094.         timex++;

```

```

1095.         if(times==20)
1096.         {
1097.             times=0;
1098.             LED1=!LED1;
1099.         }
1100.         delay_ms(10);
1101.         //sim_at_response(1);           //检查 GSM 模块发送过来的数据,及时上传给电脑
1102.     }
1103.     return 0;
1104. }
1105. //PZ-SIM800C GSM/GPRS 主测试控制部分
1106.
1107. //测试界面主 UI
1108. void sim800c_mtest_ui(u16 x,u16 y)
1109. {
1110.     u8 *p,*p1,*p2;
1111.     p=mymalloc(SRAMIN,50); //申请 50 个字节的内存
1112.     LCD_Clear(WHITE);
1113.     FRONT_COLOR=RED;
1114.     LCD_ShowFontStringMid(0,y,"PZ-SIM800C 测试程序",16,240);
1115.     LCD_ShowFontString(x,y+25,200,16,"请选择:",16,0);
1116.     LCD_ShowFontString(x,y+45,200,16,"KEY0:拨号测试",16,0);
1117.     LCD_ShowFontString(x,y+65,200,16,"KEY1:短信测试",16,0);
1118.     LCD_ShowFontString(x,y+85,200,16,"KEY2:蓝牙测试",16,0);
1119.     LCD_ShowFontString(x,y+105,200,16,"KEY_UP:GPRS 测试",16,0);
1120.     FRONT_COLOR=BLUE;
1121.     USART3_RX_STA=0;
1122.     if(sim800c_send_cmd("AT+CGMI","OK",200)==0)           //查询制造商名称
1123.     {
1124.         p1=(u8*)strstr((const char*)(USART3_RX_BUF+2),"\r\n");
1125.         p1[0]=0; //加入结束符
1126.         sprintf((char*)p,"制造商:%s",USART3_RX_BUF+2);
1127.         LCD_ShowFontString(x,y+110+25,200,16,p,16,0);
1128.         USART3_RX_STA=0;
1129.     }
1130.     if(sim800c_send_cmd("AT+CGMM","OK",200)==0) //查询模块名字
1131.     {
1132.         p1=(u8*)strstr((const char*)(USART3_RX_BUF+2),"\r\n");
1133.         p1[0]=0; //加入结束符
1134.         sprintf((char*)p,"模块型号:%s",USART3_RX_BUF+2);
1135.         LCD_ShowFontString(x,y+130+25,200,16,p,16,0);
1136.         USART3_RX_STA=0;
1137.     }
1138.     if(sim800c_send_cmd("AT+CGSN","OK",200)==0) //查询产品序列号

```

```

1139.     {
1140.         p1=(u8*)strstr((const char*)(USART3_RX_BUF+2),"\r\n");//查找回车
1141.         p1[0]=0;//加入结束符
1142.         sprintf((char*)p,"序列号:%s",USART3_RX_BUF+2);
1143.         LCD_ShowFontString(x,y+150+25,200,16,p,16,0);
1144.         USART3_RX_STA=0;
1145.     }
1146.     if(sim800c_send_cmd("AT+CNUM", "+CNUM",200)==0)           //查询本机号码
1147.     {
1148.         p1=(u8*)strstr((const char*)(USART3_RX_BUF),",");
1149.         p2=(u8*)strstr((const char*)(p1+2),"\");
1150.         p2[0]=0;//加入结束符
1151.         sprintf((char*)p,"本机号码:%s",p1+2);
1152.         LCD_ShowFontString(x,y+170+25,200,16,p,16,0);
1153.         USART3_RX_STA=0;
1154.     }
1155.     myfree(SRAMIN,p);
1156. }
1157. //GSM 信息显示(信号质量,电池电量,日期时间)
1158. //返回值:0,正常
1159. //其他,错误代码
1160. u8 sim800c_gsminfo_show(u16 x,u16 y)
1161. {
1162.     u8 *p,*p1,*p2;
1163.     u8 res=0;
1164.     p=mymalloc(SRAMIN,50);//申请 50 个字节的内存
1165.     FRONT_COLOR=BLUE;
1166.     USART3_RX_STA=0;
1167.     if(sim800c_send_cmd("AT+CPIN?", "OK",200))res|=1<<0;    //查询 SIM 卡是否在位
1168.     USART3_RX_STA=0;
1169.     if(sim800c_send_cmd("AT+COPS?", "OK",200)==0)           //查询运营商名字
1170.     {
1171.         p1=(u8*)strstr((const char*)(USART3_RX_BUF),"\");
1172.         if(p1)//有有效数据
1173.         {
1174.             p2=(u8*)strstr((const char*)(p1+1),"\");
1175.             p2[0]=0;//加入结束符
1176.             sprintf((char*)p,"运营商:%s",p1+1);
1177.             LCD_ShowFontString(x,y,200,16,p,16,0);
1178.         }
1179.         USART3_RX_STA=0;
1180.     }else res|=1<<1;
1181.     if(sim800c_send_cmd("AT+CSQ", "+CSQ:",200)==0)           //查询信号质量
1182.     {

```

```

1183.     p1=(u8*)strstr((const char*)(USART3_RX_BUF),":");
1184.     p2=(u8*)strstr((const char*)(p1),",");
1185.     p2[0]=0;//加入结束符
1186.     sprintf((char*)p,"信号质量:%s",p1+2);
1187.     LCD_ShowFontString(x,y+20,200,16,p,16,0);
1188.     USART3_RX_STA=0;
1189. }else res|=1<<2;
1190. if(sim800c_send_cmd("AT+CBC","+CBC:",200)==0) //查询电池电量
1191. {
1192.     p1=(u8*)strstr((const char*)(USART3_RX_BUF),",");
1193.     p2=(u8*)strstr((const char*)(p1+1),",");
1194.     p2[0]=0;p2[5]=0;//加入结束符
1195.     sprintf((char*)p,"电池电量:%s%% %smV",p1+1,p2+1);
1196.     LCD_ShowFontString(x,y+40,200,16,p,16,0);
1197.     USART3_RX_STA=0;
1198. }else res|=1<<3;
1199. if(sim800c_send_cmd("AT+CCLK?","+CCLK:",200)==0) //查询电池电量
1200. {
1201.     p1=(u8*)strstr((const char*)(USART3_RX_BUF),"\\"");
1202.     p2=(u8*)strstr((const char*)(p1+1),":");
1203.     p2[3]=0;//加入结束符
1204.     sprintf((char*)p,"日期时间:%s",p1+1);
1205.     LCD_ShowFontString(x,y+60,200,16,p,16,0);
1206.     USART3_RX_STA=0;
1207. }else res|=1<<4;
1208. myfree(SRAMIN,p);
1209. return res;
1210.}
1211. //NTP 更新时间
1212. void ntp_update(void)
1213. {
1214.     sim800c_send_cmd("AT+SAPBR=3,1,\"Contype\",\"GPRS\",\"OK\",200);//配置承载场景 1
1215.     sim800c_send_cmd("AT+SAPBR=3,1,\"APN\",\"CMNET\",\"OK\",200);
1216.     sim800c_send_cmd("AT+SAPBR=1,1",0,200);//激活一个 GPRS 上下文
1217.     delay_ms(5);
1218.     sim800c_send_cmd("AT+CNTPCID=1","OK",200);//设置 CNTP 使用的 CID
1219.     sim800c_send_cmd("AT+CNTP=\"202.120.2.101\",32\",\"OK\",200);//设置 NTP 服务器和本地时区(32 时
        区 时间最准确)
1220.     sim800c_send_cmd("AT+CNTP","+CNTP: 1",600);//同步网络时间
1221.
1222. }
1223. //SIM800C 主测试程序
1224. void sim800c_test(void)
1225. {

```

```

1226.    u8 key=0;
1227.    u8 timex=0;
1228.    u8 sim_ready=0;
1229.    FRONT_COLOR=RED;
1230.    LCD_ShowFontStringMid(0,30,"PZ-SIM800C 测试程序",16,240);
1231.    while(sim800c_send_cmd("AT","OK",100))//检测是否应答 AT 指令
1232.    {
1233.        LCD_ShowFontString(40,55,200,16,"未检测到模块!!!",16,0);
1234.        delay_ms(800);
1235.        LCD_Fill(40,55,200,55+16,WHITE);
1236.        LCD_ShowFontString(40,55,200,16,"尝试连接模块...",16,0);
1237.        delay_ms(400);
1238.    }
1239.    LCD_Fill(40,55,200,55+16,WHITE);
1240.    key+=sim800c_send_cmd("ATE0","OK",200);//不回显
1241.    sim800c_mtest_ui(40,30);
1242.    ntp_update();//网络同步时间
1243.    while(1)
1244.    {
1245.        delay_ms(10);
1246.        sim_at_response(1); //检查 GSM 模块发送过来的数据,及时上传给电脑
1247.        if(sim_ready)      //SIM 卡就绪.
1248.        {
1249.            key=KEY_Scan(0);
1250.            if(key)
1251.            {
1252.                switch(key)
1253.                {
1254.                    case KEY0_PRESS:
1255.                        sim800c_call_test();//拨号测试
1256.                        break;
1257.                    case KEY1_PRESS:
1258.                        sim800c_sms_test(); //短信测试
1259.                        break;
1260.                    case KEY_UP_PRESS:
1261.                        sim800c_gprs_test();//GPRS 测试
1262.                        break;
1263.                    case KEY2_PRESS:
1264.                        sim800c_spp_test();//蓝牙 spp 测试
1265.                        break;
1266.                }
1267.                sim800c_mtest_ui(40,30);
1268.                timex=0;
1269.            }

```

```

1270.     }
1271.     if(timex==0)           //2.5 秒左右更新一次
1272.     {
1273.         if(sim800c_gsminfo_show(40,225)==0)sim_ready=1;
1274.         else sim_ready=0;
1275.     }
1276.     if((timex%20)==0)LED1=!LED1;//200ms 闪烁
1277.     timex++;
1278. }
1279. }

```

此部分代码比较多，我们挑几个重要的函数进行讲解一下。

首先，是检测模块应答函数：`u8* sim800c_check_cmd(u8 *str)`，该函数用于检测 PZ-SIM800C 模块发送回来的应答/数据，其中 `str` 为期待应答字符串，返回值如果为 0，则表示没有收到期待应答字符串，否则为期待应答字符串所在的位置。

第二个函数是：`u8 sim800c_send_cmd(u8 *cmd,u8 *ack,u16 waittime)`，该函数用于向 PZ-SIM800C 模块发送命令。`cmd` 为命令字符串，当 `cmd<=0xFF` 的时候，则直接发送 `cmd`，比如短信发送结束的时候，需要发送 `0x1A`，也可以通过该函数发送。`ack` 为期待应答字符串，`waittime` 为等待时间（单位：10ms）。由于在蓝牙 SPP 测试，寻找设备模式下，扫描时需要 11S 时间后才会有返回的设备信息，所以在这函数中，我们做了 `waittime` 判断，当设置 `waittime` 等待时间等于 1100（即 11S）时，会调用 `TIM7_SetARR()` 函数，重新设置定时器的预装载周期值，设置为 1S 的更新中断，当时间累加到 11S 时，定时器才重新设置为默认的 10ms 更新中断，继续实现两个字符接收间隔以 10ms 为标准。（具体的请查看 `time.c` 中 `TIM7_IRQHandler()` 定时器 7 的中断服务函数和 `usart3.c` 中 `USART3_IRQHandler` 串口 3 的中断服务函数）

第三个函数是：`u8 sim800c_wait_request(u8 *request,u16 waittime)`，该函数用于等待 PZ-SIM800C 模块命令返回。`ack` 为期待应答字符串，`waittime` 为等待时间（单位：10ms）。该函数在蓝牙 SPP 测试中使用到。

第四个函数是：`void sim800c_unigbk_exchange(u8 *src,u8 *dst,u8 mode)`，该函数用于将输入字符串 `src` 转换为 UNICODE 编码字符串或者 GBK 内码，通过 `dst` 输出转换结果。`mode` 用于控制是 unicode 转换为 gbk（`mode=0`），还是 gbk 转换为 unicode（`mode=1`）。该函数通过调用 FATFS 提

供的 `ff_convert` 函数实现 UNICODE 码与 GBK 码转换。

第五个函数是：`u8 sim800c_call_test(void)`，该函数用于拨号测试。通过虚拟键盘（在 LCD 上触摸屏输入，下同），可以输入任意电话号码，实现拨打电话功能，并且在收到来电的时候，可以通过虚拟键盘接听/挂断来电。

第六个函数是：`void sim800c_sms_read_test(void)`，该函数用于读短信测试。该函数可以读取中英文短信，通过虚拟键盘输入要读的短信编号，即可读取短信，并且语音报读，其内容并显示在 LCD 上，还可以显示短信状态（已读/未读）、短信发送方号码、接收时间等信息。

第七个函数是：`void sim800c_sms_send_test(void)`，该函数用于发短信测试。该函数可以向任意号码，发送一条固定内容（存放在 `sim800c_test_msg`）的中英文短信。通过虚拟键盘输入电话号码，即可实现短信发送。

第八个函数是：`void sim800c_tcpudp_test(u8 mode, u8* ipaddr, u8* port)`，该函数用于 TCP/UDP 通信测试。`ipaddr` 和 `port` 分别是目标 IP 地址及其端口号，`mode` 为 0 的时候，进行 TCP 测试，`mode` 为 1 的时候进行 UDP 测试。该函数在连接成功后，就可以实现和目标 IP 地址进行 TCP/UDP 数据通信，收到的数据会显示在 LCD 上，另外也可以通过按键 `KEY0` 向目标 IP 地址发送数据。该函数还带有心跳和自动重连功能，可以实现长时间维持连接，具有很高的实用价值。

第九个函数是：`u8 sim800c_gprs_test(void)`，该函数用于 GPRS 测试。通过 `KEY_UP` 按键，可以设置连接方式（TCP/UDP），通过虚拟键盘可以输入需要连接的目标 IP 地址，端口号固定为：8086。该函数通过调用 `sim800c_tcpudp_test` 函数实现 TCP/UDP 连接测试。

第十个函数是：`u8 sim800c_spp_mode(u8 mode)`，该函数用于蓝牙不同模式连接下的 SPP 通信，`mode` 为 0 的时候进行接收配对连接，`mode` 为 1 的时候进行寻找设备连接，连接成功后，就可以实现和手机端蓝牙 app 进行数据通信了，收到的数据会显示在 LCD 上，另外模块会自动发送数据到手机端 app 上。

第十一个函数是：`u8 sim800c_spp_test(void)`，该函数用于蓝牙连接模式选择。通过按 `KEY2` 按键，设置寻找设备模式，按 `KEY0` 按键，设置为接收配对

模式,选择好模式后通过调用 `u8 sim800c_spp_mode(u8 mode)` 函数实现蓝牙 SPP 数据通信。

最后要介绍的函数是: `void sim800c_test(void)`,该函数是本 PZ-SIM800C 模块测试的主函数,该函数将在 LCD 上面显示:制造商、模块型号、序列号、本机号码、运营商、信号质量、电池电量和日期时间等参数。通过按键 KEY0,可以进入拨号测试功能;按键 KEY1,可以进入短信测试功能;按键 KEY_UP,可以进入 GPRS 测试功能;按键 KEY2,可以进入蓝牙 SPP 测试功能。

3.2 main.c

`sim800c.c` 我们就介绍到这里,我们再来看看 `main.c`,该文件里面就一个 `main` 函数, `main` 函数代码如下:

```
1.  #include "system.h"
2.  #include "SysTick.h"
3.  #include "led.h"
4.  #include "usart.h"
5.  #include "tftlcd.h"
6.  #include "key.h"
7.  #include "malloc.h"
8.  #include "sdio_sdcard.h"
9.  #include "flash.h"
10. #include "ff.h"
11. #include "fatfs_app.h"
12. #include "font_show.h"
13. #include "touch.h"
14. #include "usart3.h"
15. #include "sim800c.h"
16.
17.
18.
19. int main()
20. {
21.     u8 key;
22.
23.     SysTick_Init(168);
24.     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2); //中断优先级分组 分2组
25.     LED_Init();
26.     USART1_Init(115200);
27.     TFTLCD_Init(); //LCD 初始化
```

```

28.     KEY_Init();
29.     EN25QXX_Init();           //初始化 EN25Q128
30.     TP_Init();
31.     USART3_Init(115200);
32.     my_mem_init(SRAMIN);       //初始化内部内存池
33.     FATFS_Init();              //为 fatfs 相关变量申请内存
34.     f_mount(fs[0], "0:", 1);    //挂载 SD 卡
35.     f_mount(fs[1], "1:", 1);    //挂载 FLASH
36.     while(font_init())         //检查字库
37.     {
38.         LCD_ShowString(10,80,tftlcd_data.width,tftlcd_data.height,16, "Font Updating...");
39.         while(SD_Init())//检测不到 SD 卡
40.         {
41.             LCD_ShowString(10,80,tftlcd_data.width,tftlcd_data.height,16, "SD Card Error!");
42.             delay_ms(500);
43.             LED2=!LED2;
44.             LCD_Fill(10,80,tftlcd_data.width,80+16,BACK_COLOR);
45.         }
46.         key=update_font(10,100,16, "0:"); //更新字库
47.         while(key)//更新失败
48.         {
49.             LCD_ShowString(10,100,tftlcd_data.width,tftlcd_data.height,16, "Font Update Failed! ");
50.             delay_ms(200);
51.         }
52.         LCD_ShowString(10,100,tftlcd_data.width,tftlcd_data.height,16, "Font Update Successful!");
53.         delay_ms(1500);
54.     }
55.     key=KEY_Scan(1);
56.     if(key==KEY_UP_PRESS)
57.     {
58.         LCD_Clear(WHITE);        //清屏
59.         TP_Adjust();             //屏幕校准
60.         TP_Save_Adjdata();
61.         LCD_Clear(WHITE);        //清屏
62.     }
63.     FRONT_COLOR=RED; //设置字体为红色
64.     sim800c_test(); //GSM 测试
65. }

```

此部分代码比较简单，main 函数初始化硬件之后，由于使用到了触摸屏功能，所以开机时候可按下 KEY_UP 键强制校准，然后调用 sim800c_test 函数完成

PZ-SIM800C 模块的功能测试。

4 实验现象

首先，请先确保硬件都已经连接好了：

①给 PZ-SIM800C 模块装上 SIM 卡，并插好耳机和麦克风。

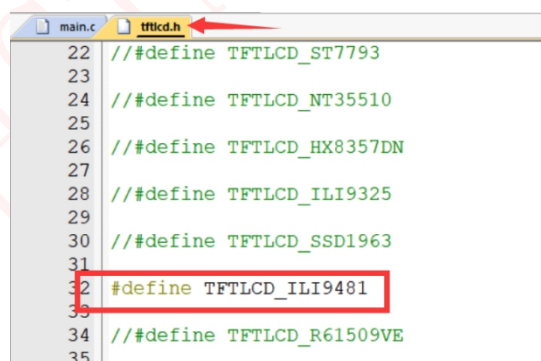
②连接 PZ-SIM800C 模块与 F407 最小系统&天马&麒麟开发板（可参考硬件设计小节）。

③给 PZ-SIM800C 模块上电（按 POWER1，电源指示灯亮）。

④PZ-SIM800C 模块开机（长按 KEY1 键开机或用跳线帽短接 P1 口的 PKEY 与 VBAT 实现上电自动开机，红色 NET1 指示灯闪烁）。

注意：如果没有 TFTLCD 触摸屏，那么只能使用串口进行 AT 指令的测试，可以参考前面 AT 指令简介章节内容。

注意：如果使用普中 TFTLCD 触摸屏，用户需根据手上彩屏右上角或背面驱动型号来设置程序中使能的 TFTLCD 驱动。在 tftlcd.h 头文件中开始处进行设置，比如我使用的 TFTLCD 是 ILI9481，那么就要把这个宏定义放开，其他的注释掉。如下所示：



```
main.c tftlcd.h
22 // #define TFTLCD_ST7793
23
24 // #define TFTLCD_NT35510
25
26 // #define TFTLCD_HX8357DN
27
28 // #define TFTLCD_ILI9325
29
30 // #define TFTLCD_SSD1963
31
32 #define TFTLCD_ILI9481
33
34 // #define TFTLCD_R61509VE
35
```

代码编译成功之后，我们将代码下载到 STM32 开发板上。（注意：以下的功能测试图片以麒麟开发板为展示）LCD 界面显示如下图所示：



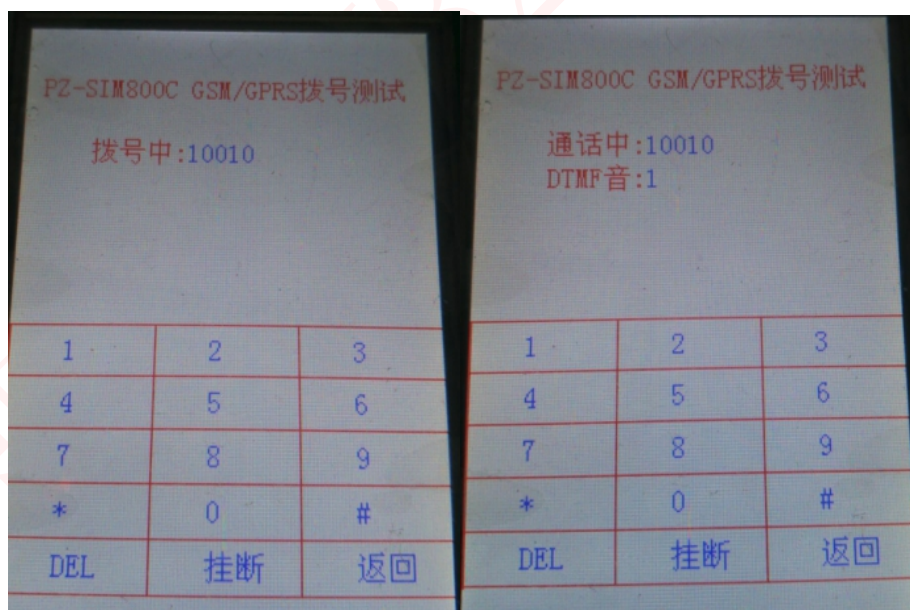
可以看到，LCD 上面显示了：制造商、模块型号、序列号、本机号码、运营商、信号质量、电池电量以及日期时间等信息。**注意：必须等到屏幕显示运营商后，才可以通过 KEY_UP/KEY0/KEY1/KEY2 这四个按键选择不同的测试项目进行测试。**

4.1 拨号测试

在主界面，按 KEY0，则可以进入此项测试，此项测试我们可以测试 PZ-SIM800C 模块的拨打电话或者接听来电等功能。拨号测试主界面如图所示：



在此界面，我们可以输入您要拨打的电话号码进行拨号。比如拨打 10010，输入 10010，然后点击“拨号”，就可以进行拨号了，如图所示：



图中，左侧图片为正在拨号中的界面，在拨号接通后，界面如图右侧图片所示，此时，我们可以通过键盘输入数字，来产生 DTMF 音，实现数字输入。比如图中我们点击数字 1，可以查询话费余额等。

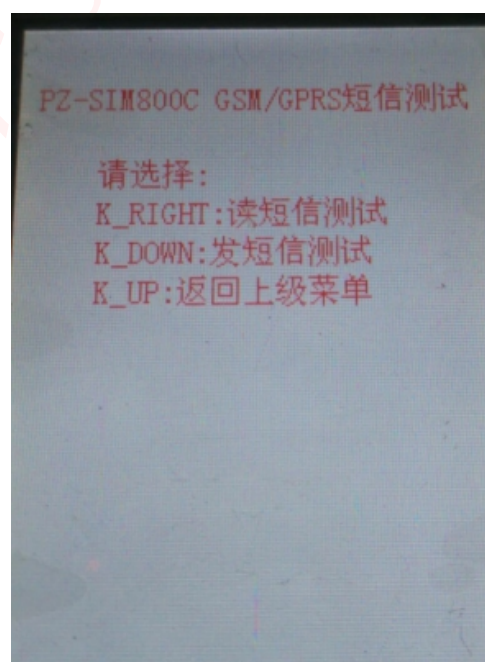
在拨号测试主界面，如果有电话接入，则会提示有来电，并显示来电电话号码，如图所示：



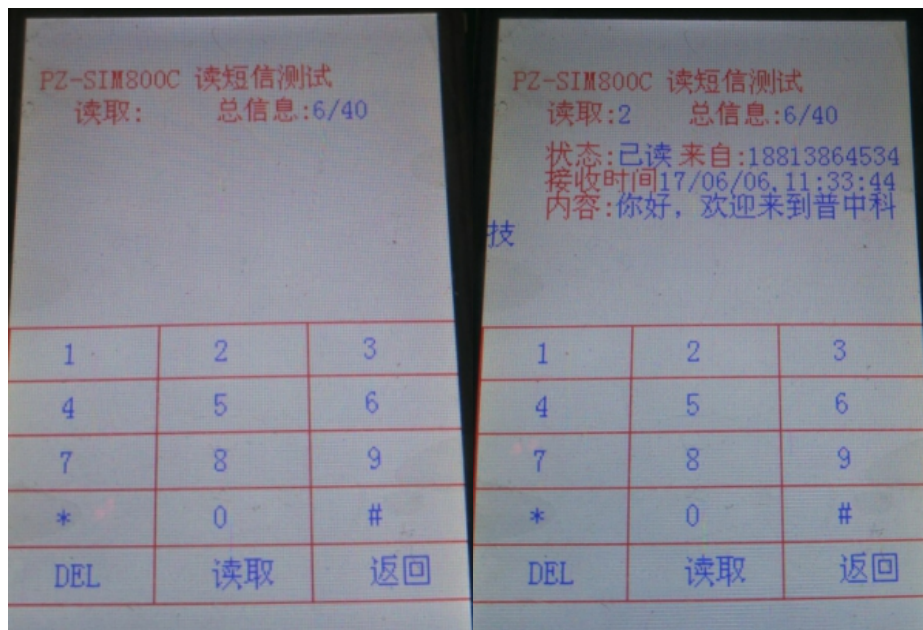
图中为来电提醒图片，此时可以在耳机听到来电铃声，我们通过点击“接听”即可接听来电，或者点击“挂断”，拒绝接听。在接通来电后，我们就可以和对方进行通话了。最后，按“返回”键，可以返回主界面。

4.2 短信测试

在主界面，按 KEY1，则可以进入此项测试，此项测试我们可以测试 PZ-SIM800C 模块的短信读取与短信发送功能。短信测试主界面如图所示：

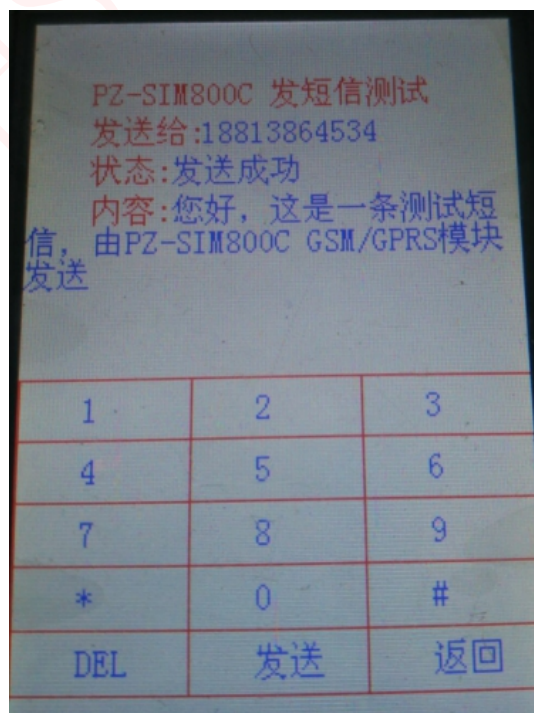


在此界面，我们按 KEY0 可以进入读短信测试，如图所示：



图中，左侧图片为刚进入读短信测试时候的界面，此时可以看到总信息提示，当前 SIM 卡中有 6 条短信，最多存储 40 条。我们通过键盘输入 2，点击“读取”，即可显示第 2 条短信，如右侧图片所示，图中不仅显示了读取到的短信内容，还显示了当前短信的状态为：已读，来自：18813864534，接收时间为：2017 年 6 月 6 号，11:33:44 等信息。

回到短信测试主界面，按 KEY1，可以进入短信发送测试，输入对方手机号码，我们就可以将一条固定内容的短信，发送到对方手机，如图所示：

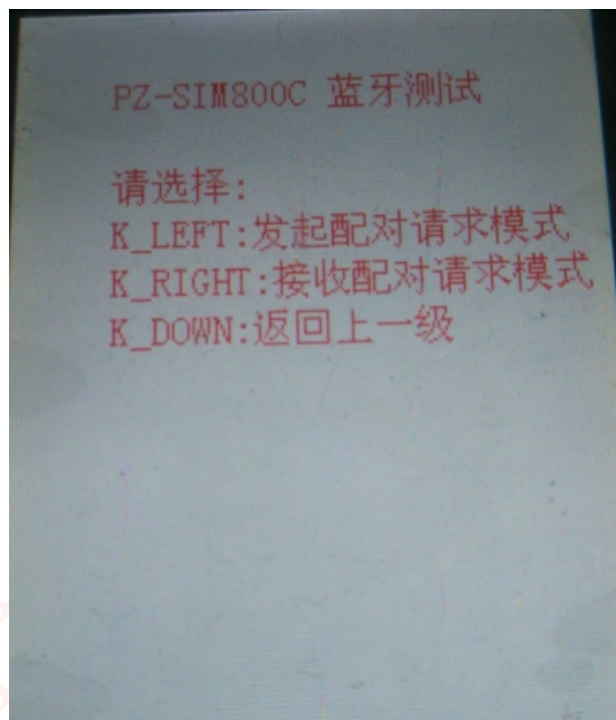


图中我们给自己发送了一条短信，左侧为短信发送时的界面，发送成功后如图片所示。

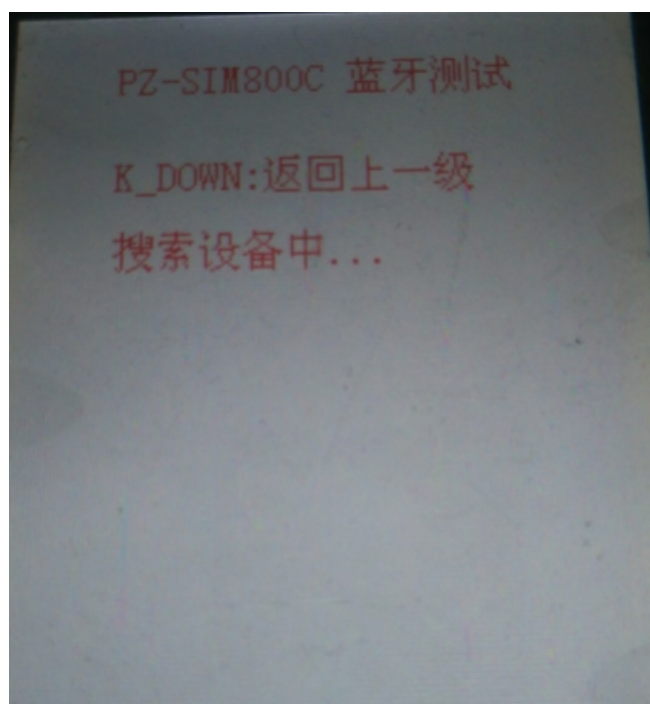
4.3 蓝牙测试

注意：由于 F407 最小系统没有 KEY2 键，用户可屏蔽一个其他功能，挪用一個按键来测试。

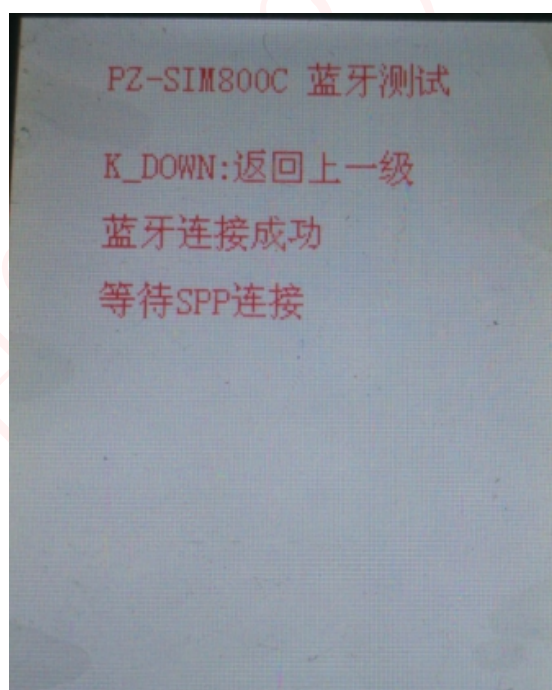
在主界面，按 KEY2，则可以进入此项测试，此项测试我们可以测试 PZ-SIM800C 模块的蓝牙通信功能。蓝牙测试主界面如图所示：



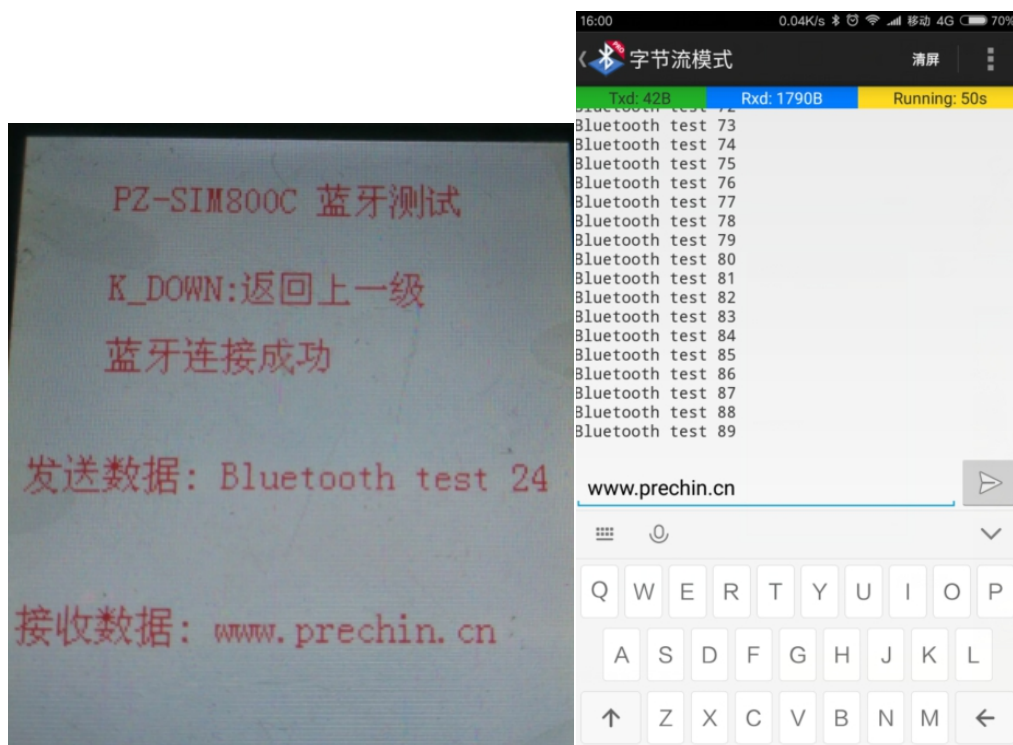
在上图所示界面，我们可以通过按键 KEY2 和 KEY0 分别选择连接的模式，我们按 KEY2 可以进入发起配对请求模式，如图所示：



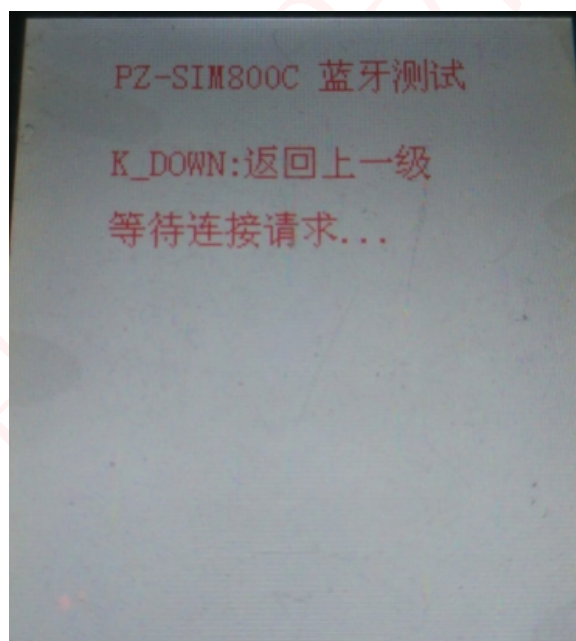
图中，模块正显示搜索设备中，这时手机端开启蓝牙设备，然后手机端接收到模块的连接请求后，配对连接成功后，这时显示等待 SPP 的连接，如图所示：



手机端打开已经安装好的蓝牙串口助手增强版 APP 这个软件（蓝牙串口助手 V0.16.apk），扫描并且连接 SIM800C 模块，连接配对成功后，手机端点击字节流模式、液晶显示屏显示模块发送给手机端的数据，而且，手机端发送的数据会在液晶显示屏上显示，如图所示：



回到蓝牙测试主界面，按 KEY0 可以进入接收配对请求模式，如图所示：



图中，模块正显示等待连接请求，手机端通过进入（设置->蓝牙），去连接可用设备 SIM800C，配对连接成功后，显示等待 spp 的连接，如上图所示，spp 的连接以及数据通信与上面的发起配对请求模式步骤和效果一样。

5 其他

(1) 购买地址（普中授权店铺）

<http://www.prechin.net/forum.php?mod=viewthread&tid=38746&extra=>

(2) 资料下载

<http://prechin.net/forum.php?mod=viewthread&tid=35264&extra=page%3D1>

(3) 技术支持

普中官网：www.prechin.cn

普中论坛：www.prechin.net

技术电话：0755-21509063（转技术）